

단원 09

인공지능소프트웨어학과

파이썬 데이터 시각화

강환수 교수



9.1 산점도와 막대 그래프

-



```
!pip install koreanize-matplotlib
import koreanize_matplotlib
%config InlineBackend.figure_format = 'retina'
```

산점도 개요와 함수 scatter()

두 연속형 변수 좌표상의 점들을 표시해 이 두 변수 사이의 관계를 나타내는 그래프

• 산점도(산포도, scatter)

- x좌표와 y좌표 (x, y)의 값이 (3, 7), (5, 10)인 2개의 점을 그리는 간단한 산점도
 - 첫 번째 인자는 두 점의 x 값만 모은 리스트이며, 두 번째가 y값 리스트 것에 주의

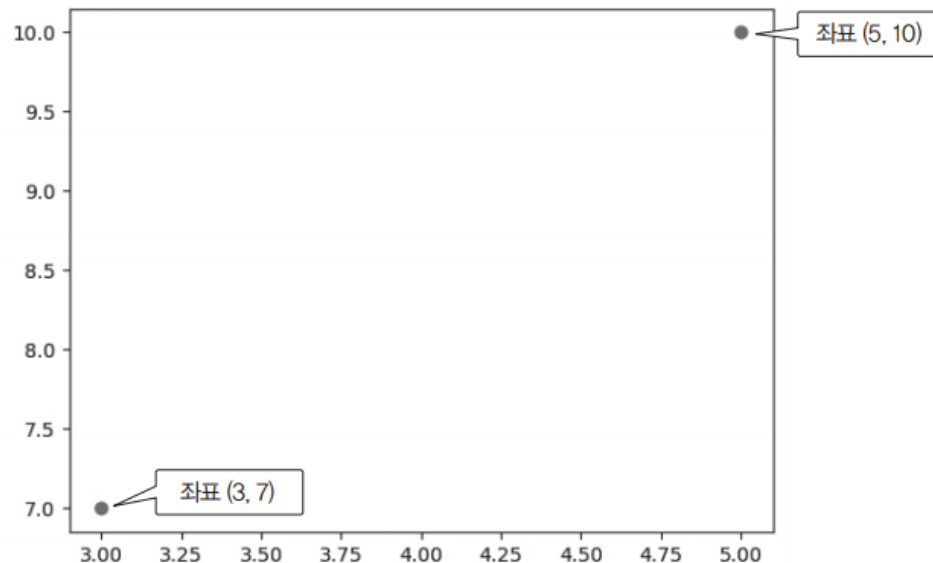
```
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize'] = (5, 3)
```

```
%config InlineBackend.figure_format = 'retina'
import matplotlib.pyplot as plt

plt.scatter([3, 5], [7, 10])
plt.show()
```

matplotlib에서 그림을 선명하게 그리기 위한 설정

x좌표와 y좌표 (x, y)의 값이 (3, 7), (5, 10)인 2개의 점을 그리는 간단한 산점도

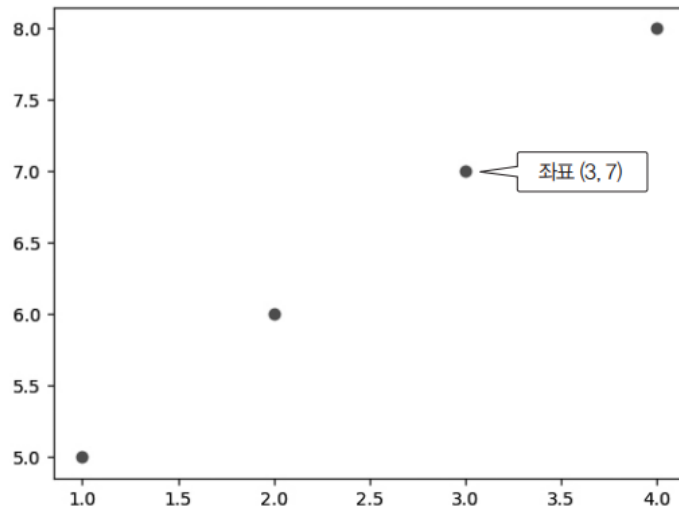


range() 결과인 수의 시퀀스로 산점도 그리기

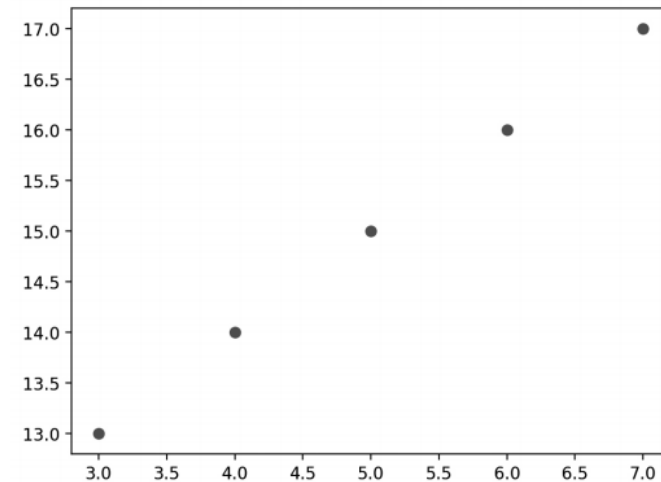
인자 x와 y의 목록 길이는 반드시 같아야 하며, 다르면 오류가 발생

```
plt.scatter(range(1, 5), range(5, 9))  
plt.show()
```

x좌표와 y좌표 (x, y)의 값이 (1, 5), (2, 6), (3, 7), (4, 8)인 4개의 점을 그리는 간단한 산점도



```
plt.scatter(range(3, 8), range(13, 18))  
plt.show()
```



내장 데이터 기니피그의 치아 성장 ToothGrowth

- 패키지 pydataset의 내장 데이터 data()에 있는 ToothGrowth에서 산점도
- ToothGrowth는 60개의 관측 값과 3개의 변수
 - 동물 기니피그의 치아성장 세포에 대한 비타민 C의 영향을 설명한 자료

열 이름	타입	설명
len	numeric	치아 길이 (mm 단위)
supp (Supplement type)	object	보충제 종류 - VC: Vitamin C (비타민C 용액) - OJ: Orange Juice (오렌지 주스)
dose (dose level)	numeric	용량 (mg/day) - 0.5, 1.0, 2.0 세 수준

```
from pydataset import data
```

```
tg = data('ToothGrowth')  
tg.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
Int64Index: 60 entries, 1 to 60  
Data columns (total 3 columns):  
#   Column    Non-Null Count  Dtype  
---  ---  
0   len       60 non-null    float64  
1   supp      60 non-null    object  
2   dose      60 non-null    float64  
dtypes: float64(2), object(1)  
memory usage: 1.9+ KB
```

두 변수 dose(비타민 용량), len(치아세포 길이)의 상관 관계를 산점도

비타민 용량이 증가할수록 치아세포 길이

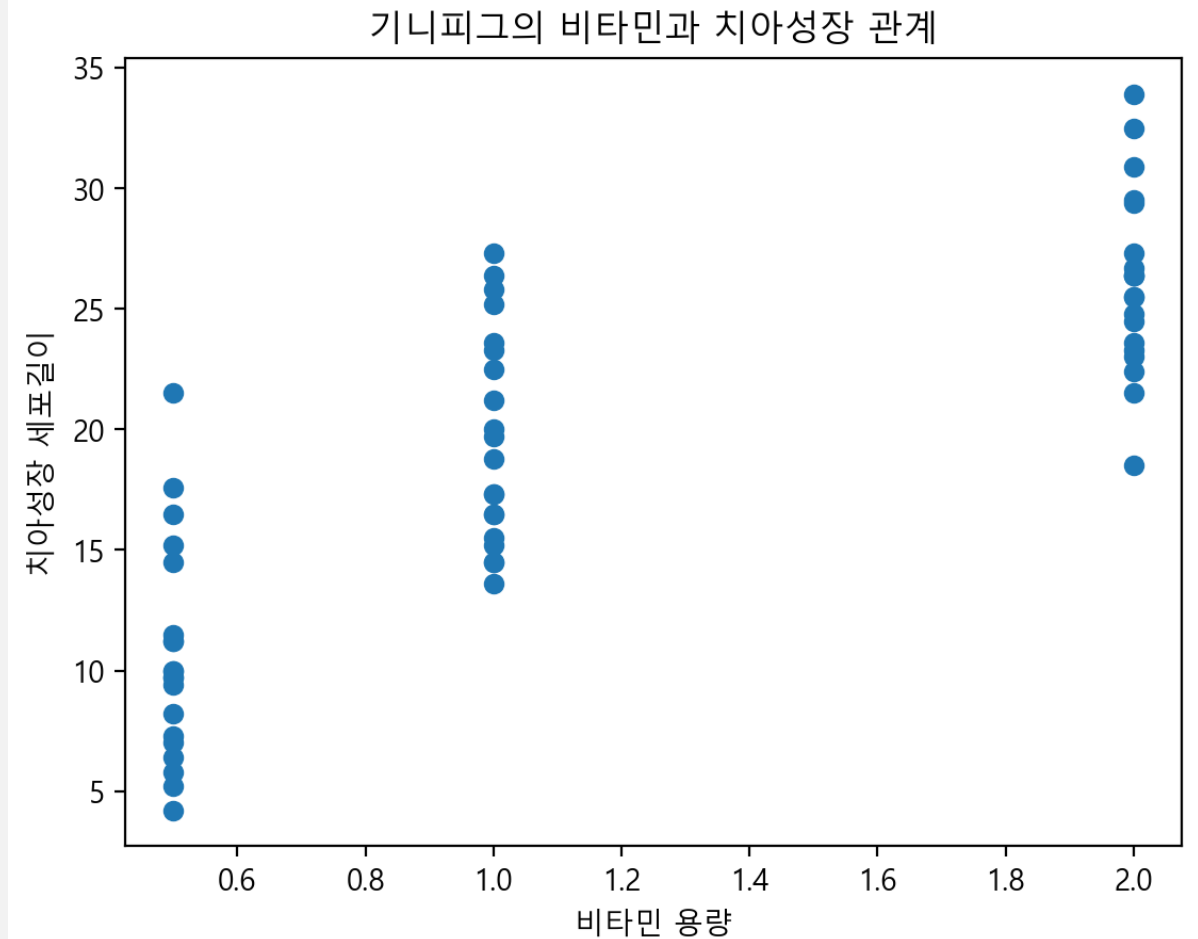
- Colab에서는 koreanize-matplotlib 사용

```
import matplotlib.pyplot as plt
# 한글 처리를 위한 코드
plt.rc('font', family='Malgun Gothic') # 폰트 지정
plt.rc('axes', unicode_minus=False) # 음수 - 표시
# plt.rcParams['font.family'] = 'Malgun Gothic'
# plt.rcParams['axes.unicode_minus'] = False

✓ 0.0s

plt.scatter(tg.dose, tg.len)
plt.title("기니피그의 비타민과 치아성장 관계")
plt.xlabel("비타민 용량")
plt.ylabel("치아성장 세포길이")
plt.show()

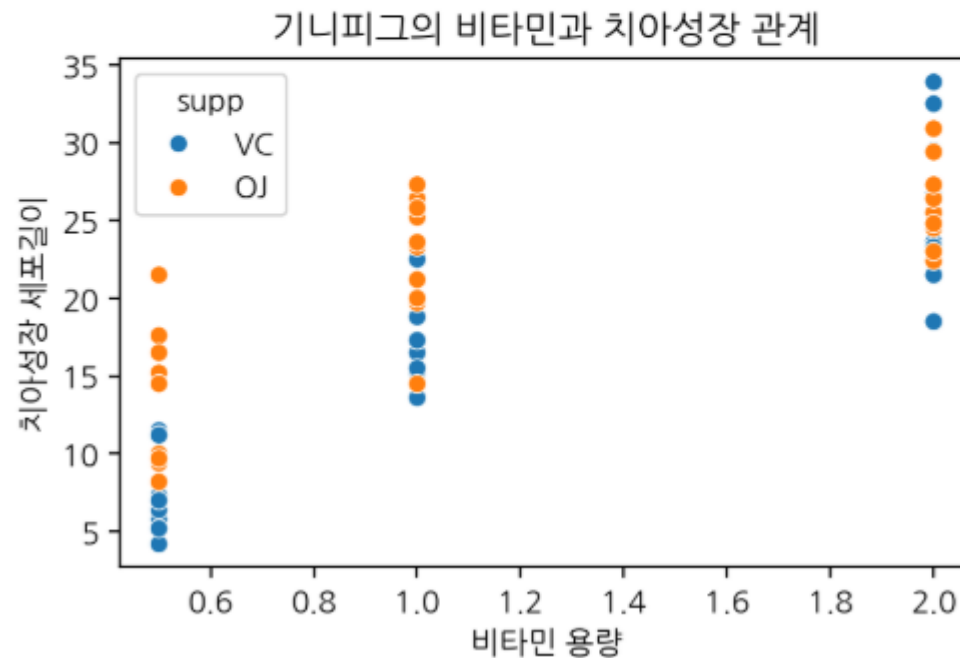
✓ 0.2s
```



비타민 종류와 용량에 따른 치아 성장 차이

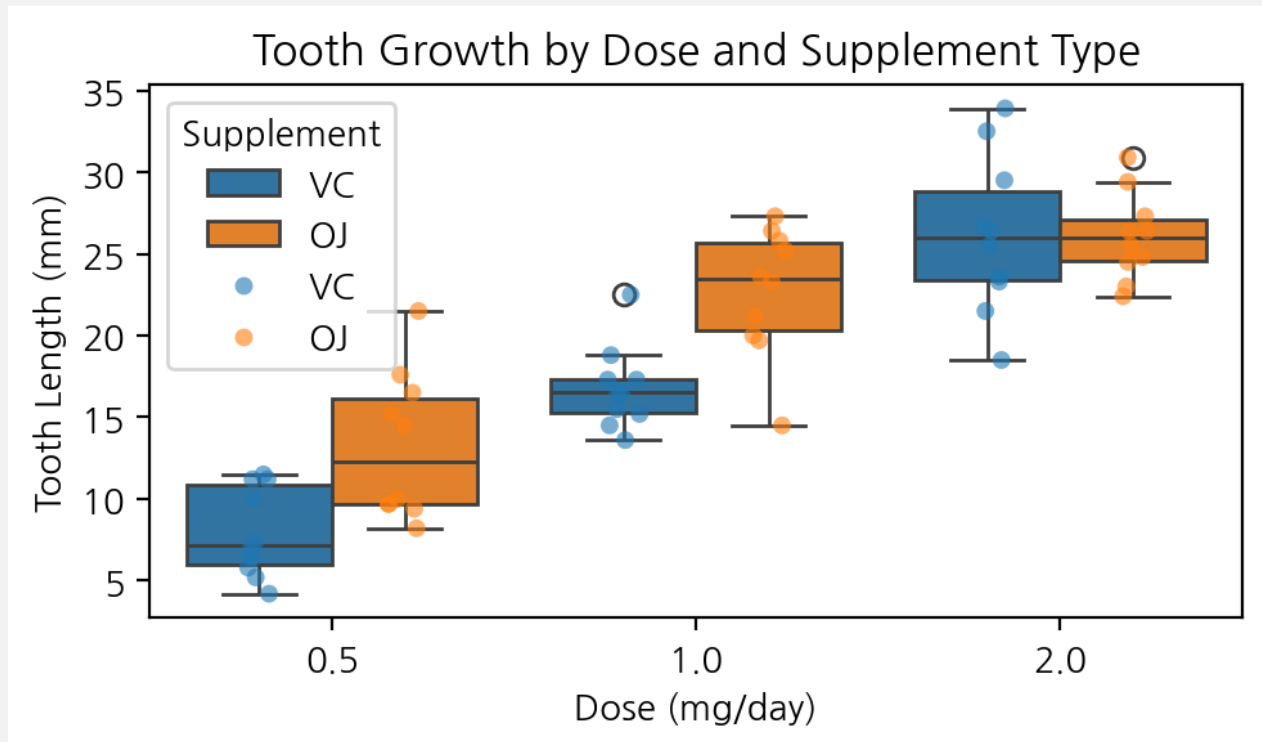
- 용량이 2mg이면 더 성장
- 용량이 0.5, 1mg에서는 오렌지 주스가 더 성장에 효과적

```
import seaborn as sns
sns.scatterplot(data=tg, x='dose', y='len', hue='supp') # 자동 색상 구분
plt.title("기니피그의 비타민과 치아성장 관계")
plt.xlabel("비타민 용량")
plt.ylabel("치아성장 세포길이")
plt.show()
```



상자 그림

```
# 시각화
sns.boxplot(data=tg, x='dose', y='len', hue='supp')
sns.stripplot(data=tg, x='dose', y='len', hue='supp',
dodge=True, alpha=0.6)
# sns.stripplot(data=tg, x='dose', y='len', hue='supp',
alpha=0.6)
plt.title('Tooth Growth by Dose and Supplement Type')
plt.ylabel('Tooth Length (mm)')
plt.xlabel('Dose (mg/day)')
plt.legend(title='Supplement')
plt.tight_layout()
plt.show()
```

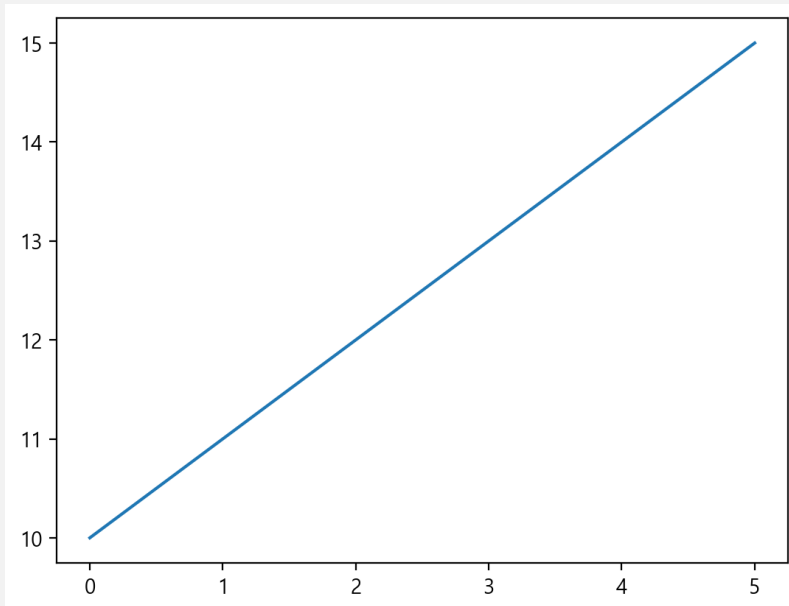


선 그리기 함수 plot()

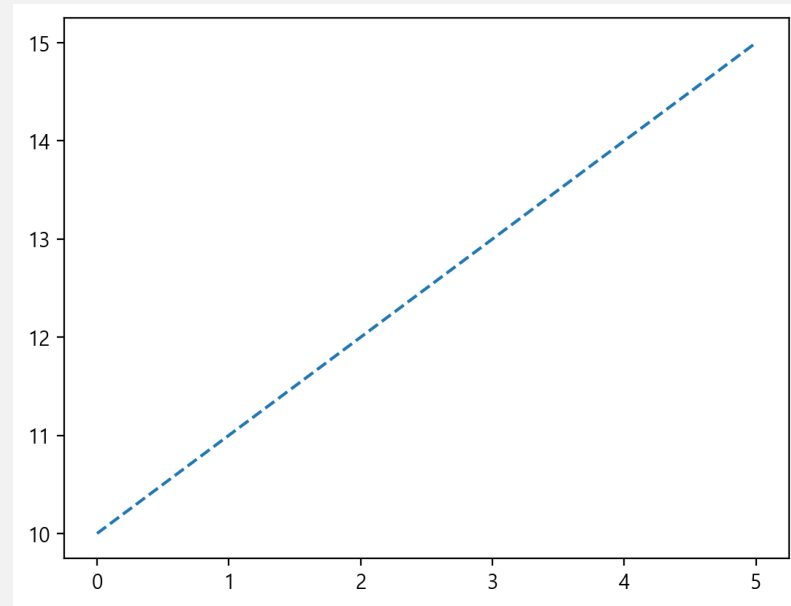
함수 plot()으로 간단히 지정한 좌표를 지나는 선 그리기

- 선 종류는 인자 `linestyle` 또는 간단히 `ls`로 지정
 - `plot([x], y, linestyle='--', ...)` 인자
 - 점선(dashed line) 표시
- 왼쪽 결과에서 선 종류가 기본인 실선
 - 오른쪽은 지정한 대시(--)

```
import matplotlib.pyplot as plt
plt.plot(range(10, 16));
```



```
plt.plot(range(10, 16), linestyle='--');
```



x축 -5에서 5까지의 그래프 $y = x^2$ 을 그린 그림

x축에서 이웃한 좌표의 거리를 0.1로 지정

- 0.1을 더 작게 하면 곡선이 더 부드러워짐

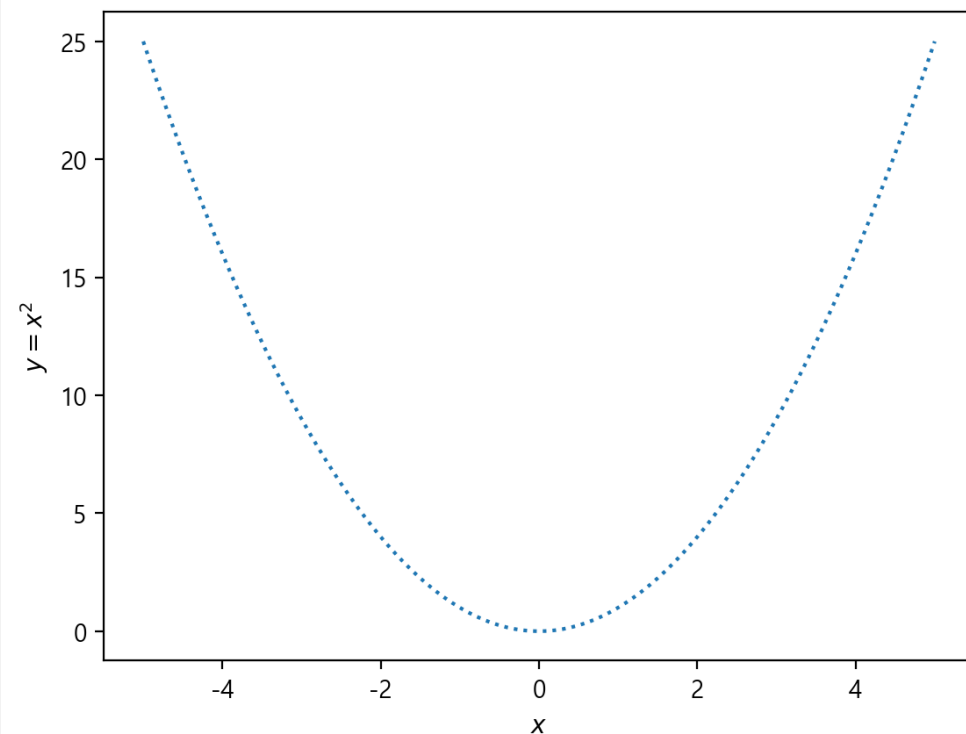
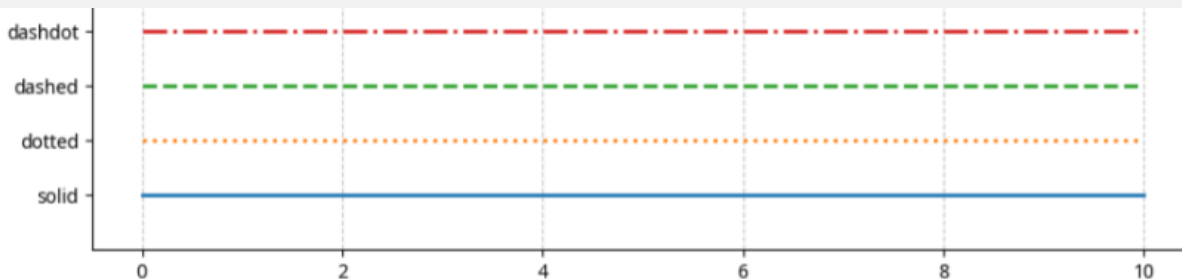
- 실수의 나열을 위해 numpy의 `arange()` 함수를 활용
- 인자 `ls`는 `linestyle`와 동일
 - ":"은 점선(dotted line)

- 종류, 4가지 기본 스타일

- `solid(-)`
- `dotted(:)`
- `dashed(--)`
- `dashdot(-.)`
- `None(", ' ')`: 선 없음

```
import numpy as np
x = np.arange(-5, 5.1, 0.1)
plt.plot(x, x**2, ls=":");
plt.xlabel('$x$');
plt.ylabel('$y = x^2$');
```

레이블에서 앞 뒤의 \$은 내용을 레이텍(lay tech) 형식의 수식으로 표현한다.



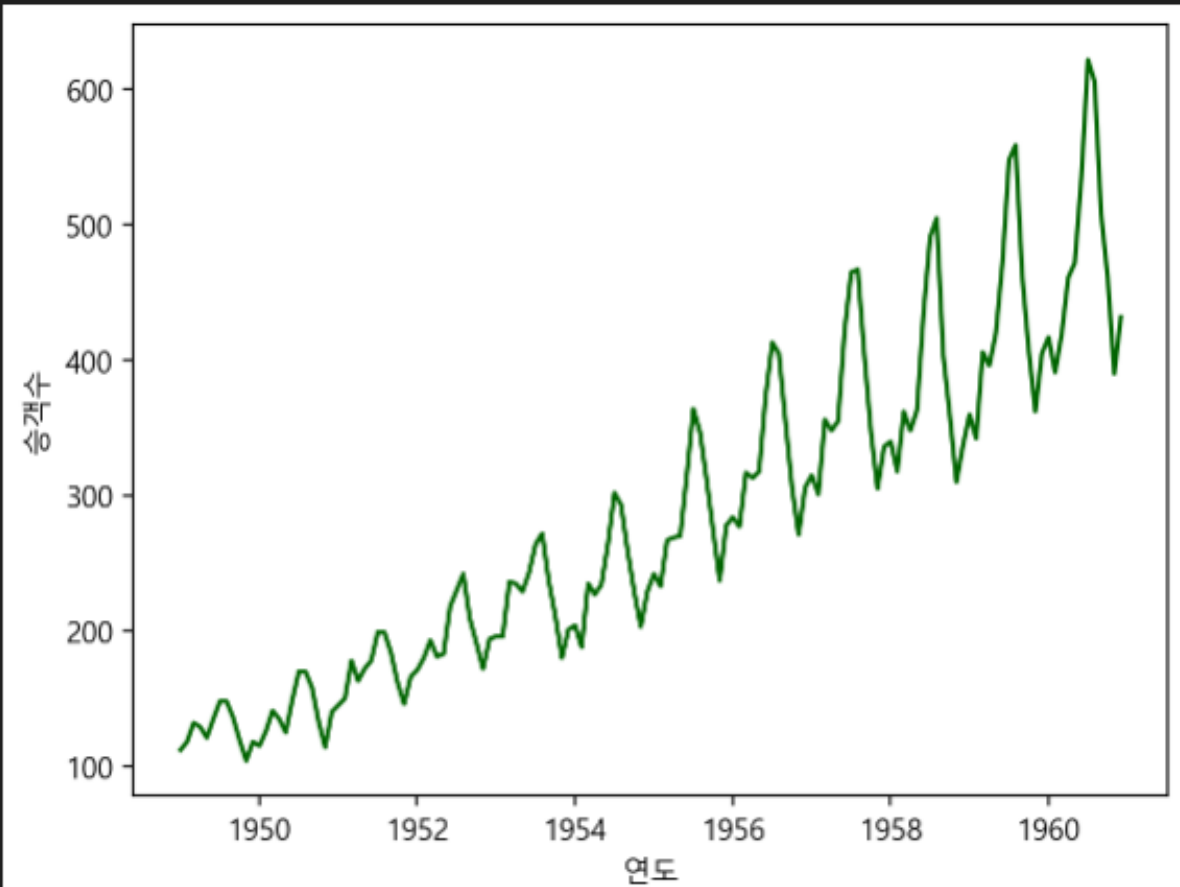
내장 데이터 AirPassengers

1949년부터 1960년까지 월별 국제선 승객 자료

```
airpsngr = data('AirPassengers')
airpsngr.info()
<class 'pandas.core.frame.DataFrame'>
Int64Index: 144 entries, 1 to 144
Data columns (total 2 columns):
#   Column          Non-Null Count  Dtype
---  -
0   time            144 non-null   float64
1   AirPassengers    144 non-null   int64
dtypes: float64(1), int64(1)
memory usage: 3.4 KB
```

```
plt.plot(airpsngr.time, airpsngr.AirPassengers, color = 'darkgreen');
plt.xlabel('연도')
plt.ylabel('승객수');
```

✓ 0.1s

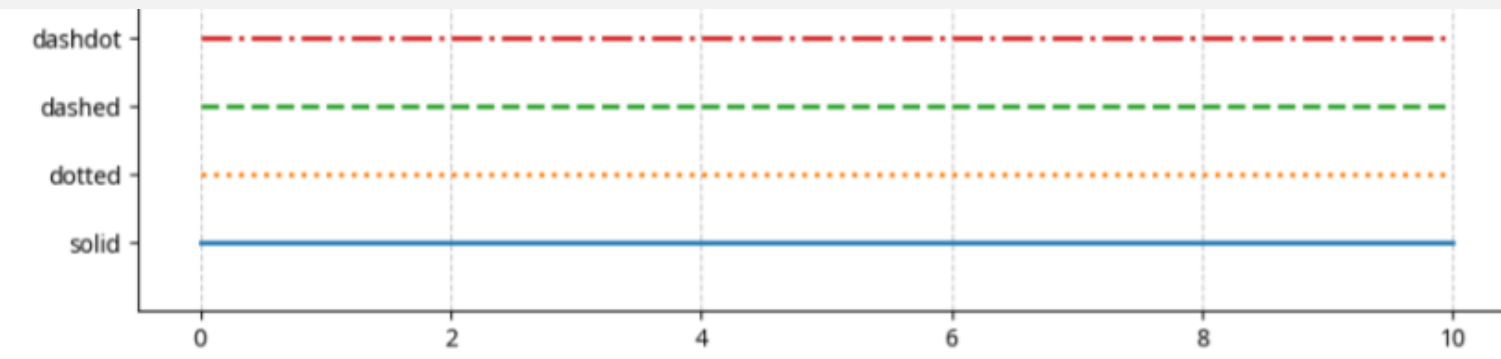


함수 plot()의 패러미터 linestyle과 marker

- 함수 `plt.plot(x, y, linestyle='-')`
 - `linestyle`(간단히 ls)
 - 선 종류를 지정하는 패러미터
 - 지정하지 않으면
 - 기본적으로 선 종류는 실선인 `"-"`
- 지정 가능한 선 종류는 다음 4가지

[표 9-1] 함수 plot()의 선 종류 패러미터 linestyle

선 스타일 이름 linestyle	선 스타일 기호	예제 코드
실선(Solid line)	'—'	<code>plt.plot(x, y, linestyle='—')</code>
점선(Dashed line)	'--'	<code>plt.plot(x, y, linestyle='--')</code>
점선(Dotted line)	'.'	<code>plt.plot(x, y, linestyle='.')'</code>
대시 점선(Dash-dot line)	'-.'	<code>plt.plot(x, y, linestyle='-.'</code>



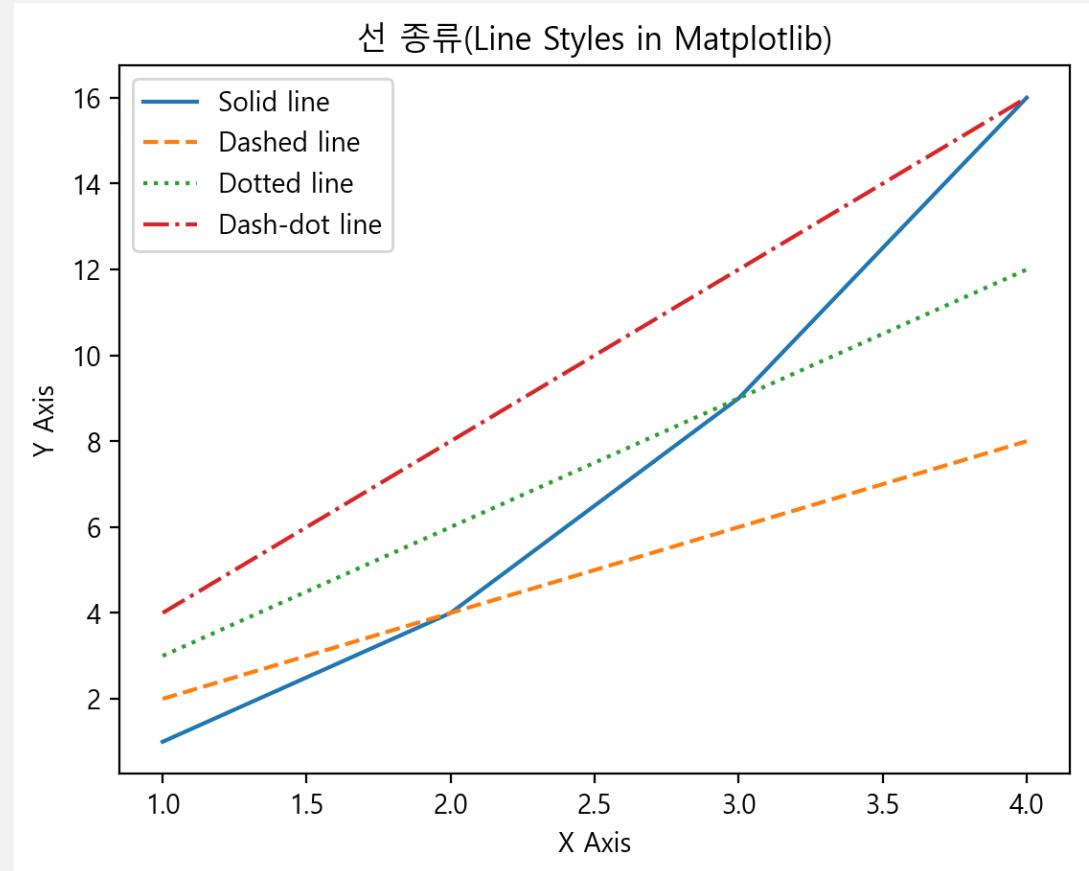
다양한 선 종류(linestyle(간단히 ls)) 결과

```
import matplotlib.pyplot as plt

# 데이터 준비
x = [1, 2, 3, 4]
y1 = [1, 4, 9, 16]
y2 = [2, 4, 6, 8]
y3 = [3, 6, 9, 12]
y4 = [4, 8, 12, 16]
# 선 스타일별로 그래프 그리기
plt.plot(x, y1, linestyle='-', label='Solid line')      # 실선
plt.plot(x, y2, linestyle='--', label='Dashed line')   # 점선
plt.plot(x, y3, linestyle=':', label='Dotted line')    # 도트 점선
plt.plot(x, y4, linestyle='-.', label='Dash-dot line') # 대시 도트 점선

# 그래프 제목과 축 레이블
plt.title("선 종류(Line Styles in Matplotlib)")
plt.xlabel("X Axis")
plt.ylabel("Y Axis")

# 범례 추가
plt.legend()
# 그래프 보여주기
plt.show()
```



마커 종류

마커(marker)를 사용하여 각 데이터 포인트를 강조

[표 9-2] 마커 종류 인자 marker

마커 이름 marker	마커 기호	예제 코드
점 (Point)	'.'	<code>plt.plot(x, y, marker='.')</code>
픽셀 (Pixel)	','	<code>plt.plot(x, y, marker=',')</code>
원 (Circle)	'o'	<code>plt.plot(x, y, marker='o')</code>
삼각형 (Triangle_down)	'v'	<code>plt.plot(x, y, marker='v')</code>
역삼각형 (Triangle_up)	'^'	<code>plt.plot(x, y, marker='^')</code>
오른쪽 삼각형 (Triangle_right)	'>'	<code>plt.plot(x, y, marker='>')</code>
왼쪽 삼각형 (Triangle_left)	'<'	<code>plt.plot(x, y, marker='<')</code>
사각형 (Square)	's'	<code>plt.plot(x, y, marker='s')</code>
다이아몬드 (Diamond)	'D'	<code>plt.plot(x, y, marker='D')</code>
작은 다이아몬드 (Thin diamond)	'd'	<code>plt.plot(x, y, marker='d')</code>
육각형1 (Hexagon1)	'h'	<code>plt.plot(x, y, marker='h')</code>
육각형2 (Hexagon2)	'H'	<code>plt.plot(x, y, marker='H')</code>
별 (Star)	'*'	<code>plt.plot(x, y, marker='*')</code>
십자 (Plus)	'+'	<code>plt.plot(x, y, marker='+')</code>
곱하기 (X)	'x'	<code>plt.plot(x, y, marker='x')</code>
작은 십자 (Thin plus)	' '	<code>plt.plot(x, y, marker=' ')</code>
작은 곱하기 (Thin x)	'_'	<code>plt.plot(x, y, marker='_')</code>

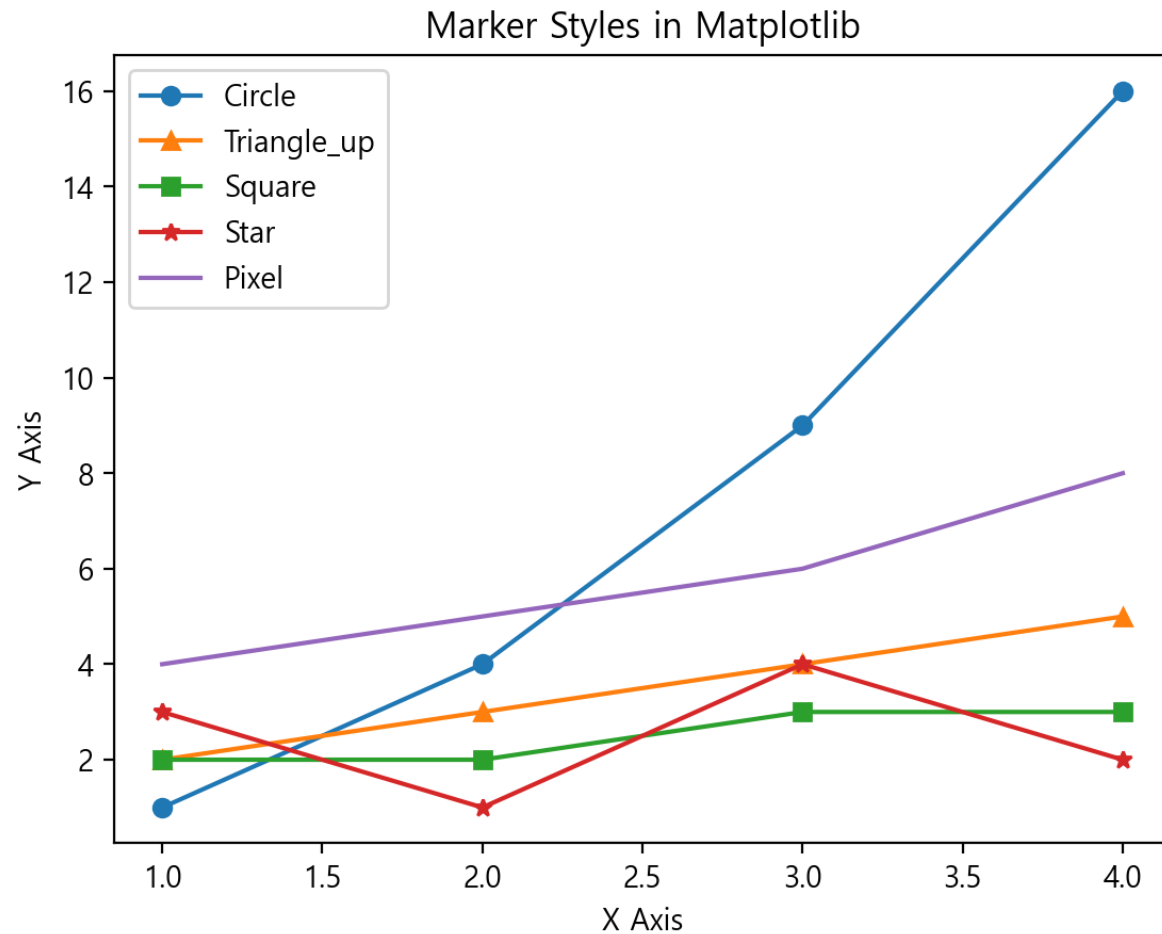
5개 마커 사용

```
import matplotlib.pyplot as plt

# 데이터 준비
x = [1, 2, 3, 4]
y1 = [1, 4, 9, 16]
y2 = [2, 3, 4, 5]
y3 = [2, 2, 3, 3]
y4 = [3, 1, 4, 2]
y5 = [4, 5, 6, 8]

# 그래프 그리기
plt.plot(x, y1, marker='o', label='Circle') # 원
plt.plot(x, y2, marker='^', label='Triangle_up') # 삼각형 위
plt.plot(x, y3, marker='s', label='Square') # 사각형
plt.plot(x, y4, marker='*', label='Star') # 별
plt.plot(x, y5, marker=',', label='Pixel') # 픽셀

# 그래프 제목과 축 레이블
plt.title("Marker Styles in Matplotlib")
plt.xlabel("X Axis")
plt.ylabel("Y Axis")
# 범례 추가
plt.legend()
# 그래프 보여주기
plt.show()
```

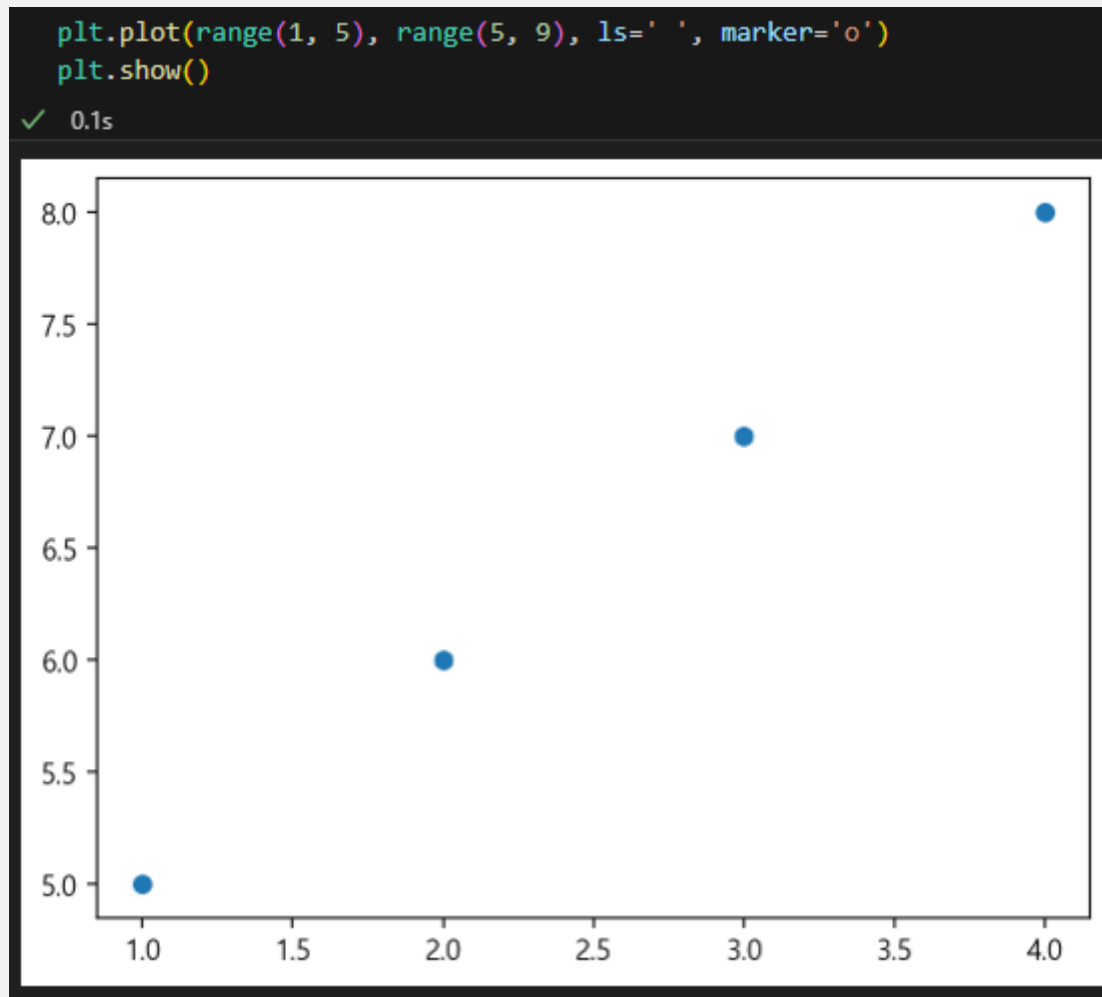


인자 `ls=' '` 또는 `ls=''` 지정

선이 없이 마커만 표시

- 함수 `plot()`

- 인자 `ls=' '`, `marker='o'`로 `scatter()`와 유사한 그림 그리기



비타민 용량과 세포길이

비타민 C의 종류가 각각 "VC"와 "OJ"인 행 중에서 열 dose와 len만으로 구성된 데이터프레임을 추출

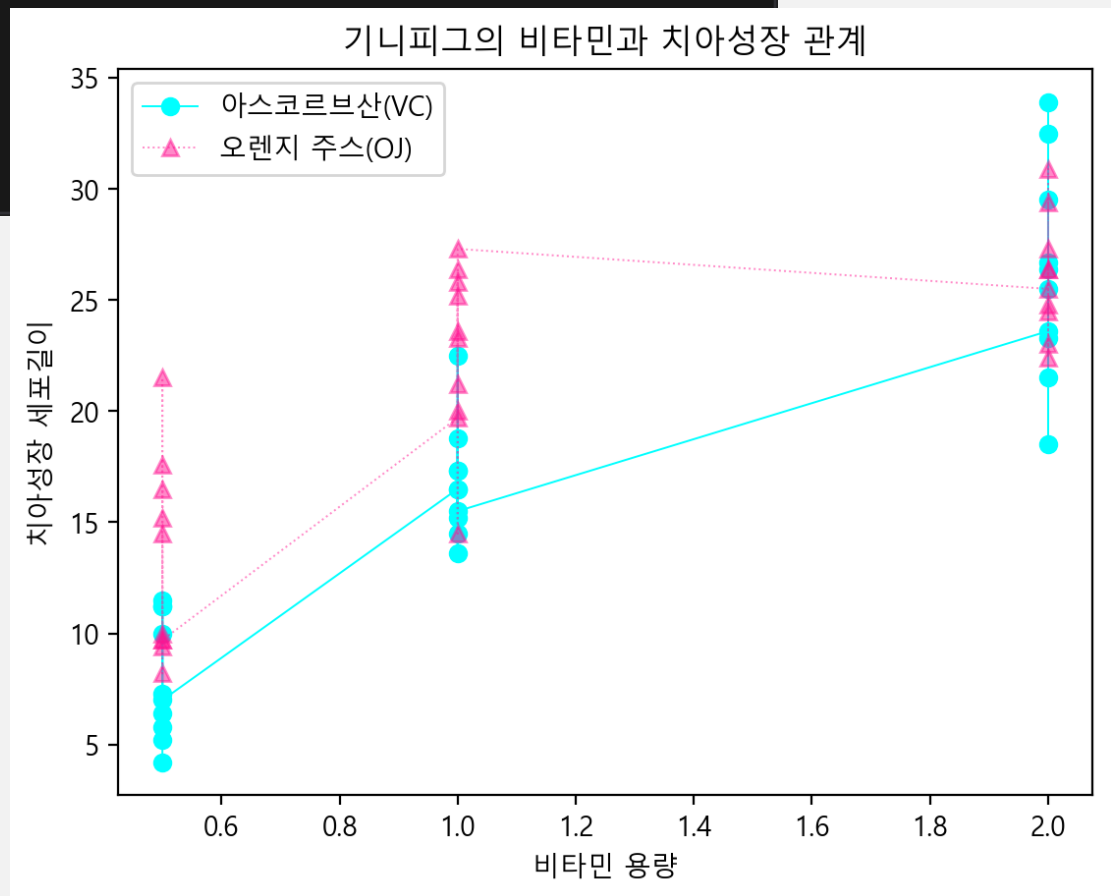
```
vc = tg[tg.suppl == "VC"][['len', 'dose']]
oj = tg[tg.suppl == "OJ"][['len', 'dose']]
```

✓ 0.0s

Python

```
plt.plot(vc.dose, vc.len, color='cyan', marker='o', label="아스코르브산(VC)", lw=.7)
plt.plot(oj.dose, oj.len, color='deeppink', marker='^', ls=':', label="오렌지 주스(OJ)", lw=.7, alpha=.5)
plt.title("기니피그의 비타민과 치아성장 관계")
plt.xlabel("비타민 용량")
plt.ylabel("치아성장 세포길이")
plt.legend();
```

✓ 0.2s



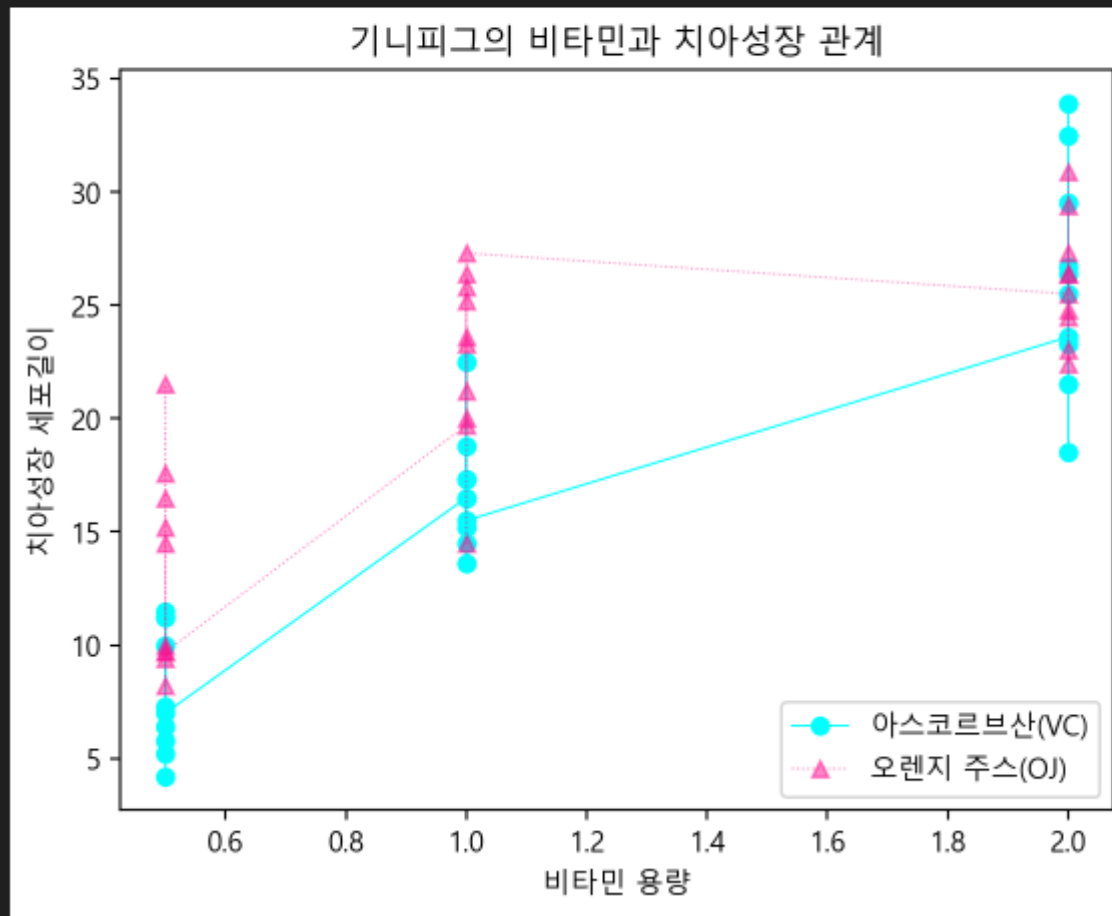
범례의 좌측 상단 모서리 위치를 좌표로 지정

```
plt.legend(loc='lower right');
```

```
plt.plot(vc.dose, vc.len, color='cyan', marker='o', label="아스코르브산(VC)", lw=.7)
plt.plot(oj.dose, oj.len, color='deeppink', marker='^', ls=':', label="오렌지 주스(OJ)", lw=.7, alpha=.5)
plt.title("기니피그의 비타민과 치아성장 관계")
plt.xlabel("비타민 용량")
plt.ylabel("치아성장 세포길이")
plt.legend(loc='lower right');
```

✓ 0.2s

Python



범례 위치 지정 방법

- 범례 위치를 간단히 지정하는 인자 `loc`의 값
 - 인자 `loc`을 지정하지 않으면 'best'

[표 9-3] 범례의 위치 지정 패러미터 `loc` 종류

위치 이름	loc 값	예제 코드
Best	'best'	<code>plt.legend(loc='best')</code>
Upper right	'upper right'	<code>plt.legend(loc='upper right')</code>
Upper left	'upper left'	<code>plt.legend(loc='upper left')</code>
Lower left	'lower left'	<code>plt.legend(loc='lower left')</code>
Lower right	'lower right'	<code>plt.legend(loc='lower right')</code>
Right	'right'	<code>plt.legend(loc='right')</code>
Center left	'center left'	<code>plt.legend(loc='center left')</code>
Center right	'center right'	<code>plt.legend(loc='center right')</code>
Lower center	'lower center'	<code>plt.legend(loc='lower center')</code>
Upper center	'upper center'	<code>plt.legend(loc='upper center')</code>
Center	'center'	<code>plt.legend(loc='center')</code>

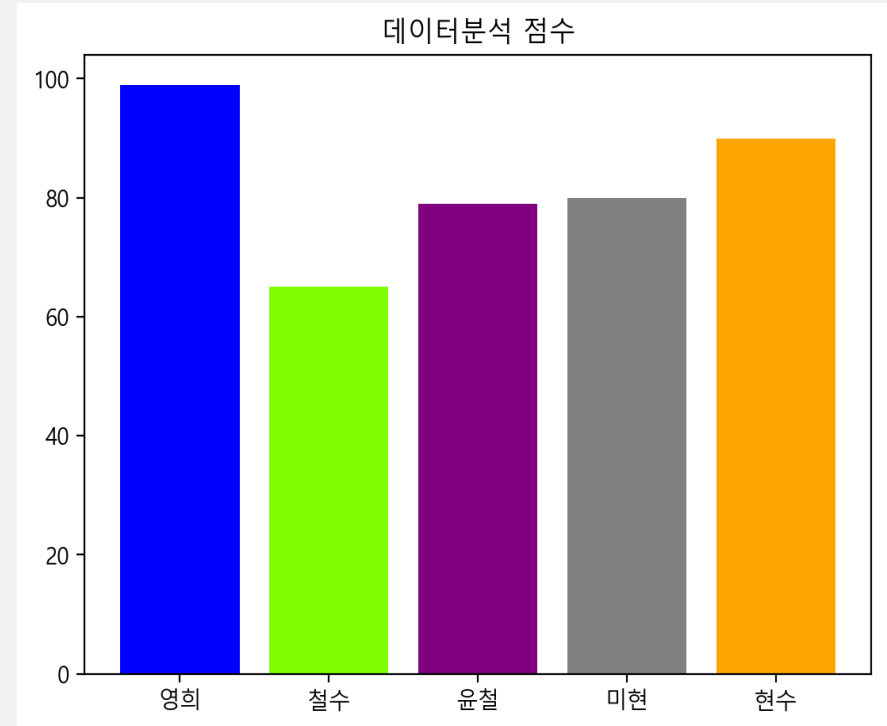
막대 그래프 함수 bar()

이산형 자료의 값을 직사각형 막대를 사용하여 데이터를 시각화하는 방법

- 함수 `bar(name, ...)` : 세로 막대 방향

- 첫 인자 `name`
 - X 축의 학생 이름인 `name`
 - 점수의 해당 학생 이름을 `name`에 저장
- 두번째 인자 `point`
 - 점수를 나타내는 막대의 세로 높이를 `point` 저장
- 인자 `color`
 - 각 막대의 색상을 지정

```
import matplotlib.pyplot as plt
point = [99, 65, 79, 80, 90]
name = ["영희", "철수", "윤철", "미현", "현수"]
plt.bar(name, point, color = ["blue", "chartreuse", "purple", "grey", "orange"])
plt.title('데이터분석 점수');
```



막대 그래프 함수 barh()

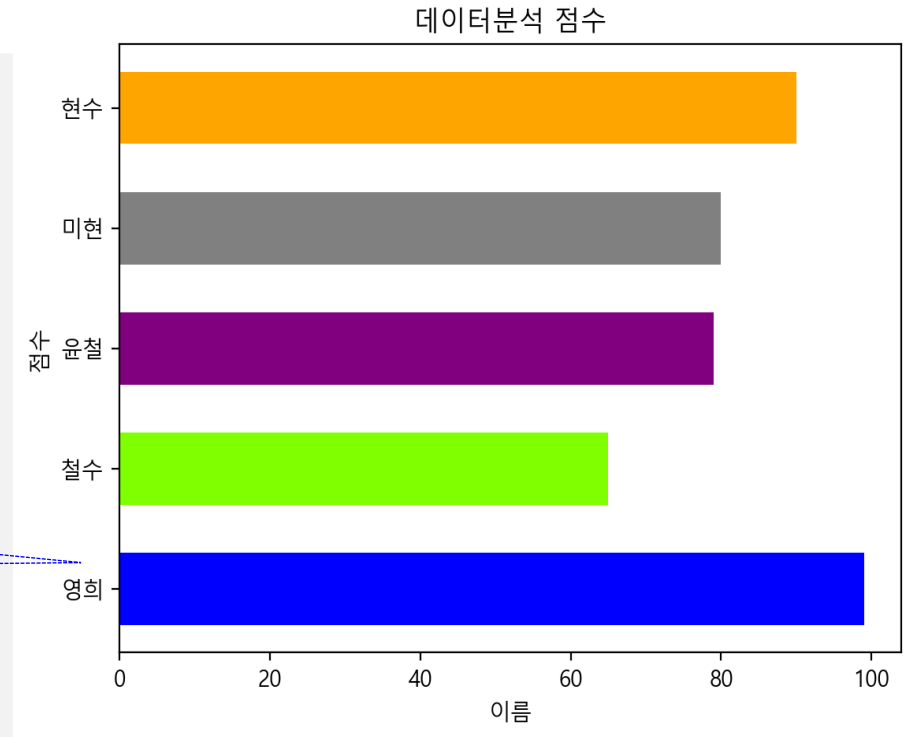
가로 막대 그래프

- 함수 barh()

- 막대 그래프를 가로로 그리기
- 인자 height=.6(60%)으로 막대의 너비를 조절

벡터 내용이 아래부터 그려진다.

```
import matplotlib.pyplot as plt
point = [99, 65, 79, 80, 90]
name = ["영희", "철수", "윤철", "미현", "현수"]
plt.barh(name, point, height = .6,
          color = ["blue", "chartreuse", "purple", "grey", "orange"])
plt.title('데이터분석 점수')
plt.xlabel('이름')
plt.ylabel('점수');
```



내장 데이터 LifeCycleSavings

“국가별 생애주기 저축 데이터”로 5개의 변수와 50개의 관측 값

- 1960년부터 1970년까지의 10년 평균 자료

“15세 미만의 인구비율”을
나타내는 열 pop15

```
df = lcs[:5][['pop15']]
df
```

✓ 0.0s

	pop15
Australia	29.35
Austria	23.32
Belgium	23.80
Bolivia	41.89
Brazil	42.19

```
from pydataset import data
lcs = data('LifeCycleSavings')
lcs.info()
```

✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>
Index: 50 entries, Australia to Malaysia
Data columns (total 5 columns):
#   Column  Non-Null Count  Dtype
---  -
0    sr      50 non-null       float64
1   pop15   50 non-null       float64
2   pop75   50 non-null       float64
3    dpi     50 non-null       float64
4   ddpi    50 non-null       float64
dtypes: float64(5)
memory usage: 2.3+ KB
```

lcs.describe()

✓ 0.0s

	sr	pop15	pop75	dpi	ddpi
count	50.000000	50.000000	50.000000	50.000000	50.000000
mean	9.671000	35.089600	2.293000	1106.758400	3.757600
std	4.480407	9.151727	1.290771	990.868889	2.869871
min	0.600000	21.440000	0.560000	88.940000	0.220000
25%	6.970000	26.215000	1.125000	288.207500	2.002500
50%	10.510000	32.575000	2.175000	695.665000	3.000000
75%	12.617500	44.065000	3.325000	1795.622500	4.477500
max	21.100000	47.640000	4.700000	4001.890000	16.710000

데이터 LifeCycleSavings 열 설명

"Disposable Personal Income" (가처분 개인소득): 세후 개인소득

열 이름	데이터 타입	설명	상세 의미
sr	numeric	**Aggregate personal savings**	가처분 소득 대비 총 개인 저축액의 비율 (저축률)
pop15	numeric	**% of population under 15**	15세 미만 인구가 전체 인구에서 차지하는 비율 (%)
pop75	numeric	**% of population over 75**	75세 초과 인구가 전체 인구에서 차지하는 비율 (%)
dpi	numeric	**Real per-capita disposable income**	실질 1인당 가처분 소득 (달러 단위)
ddpi	numeric	**% growth rate of dpi**	1인당 가처분 소득의 연평균 성장률 (%)

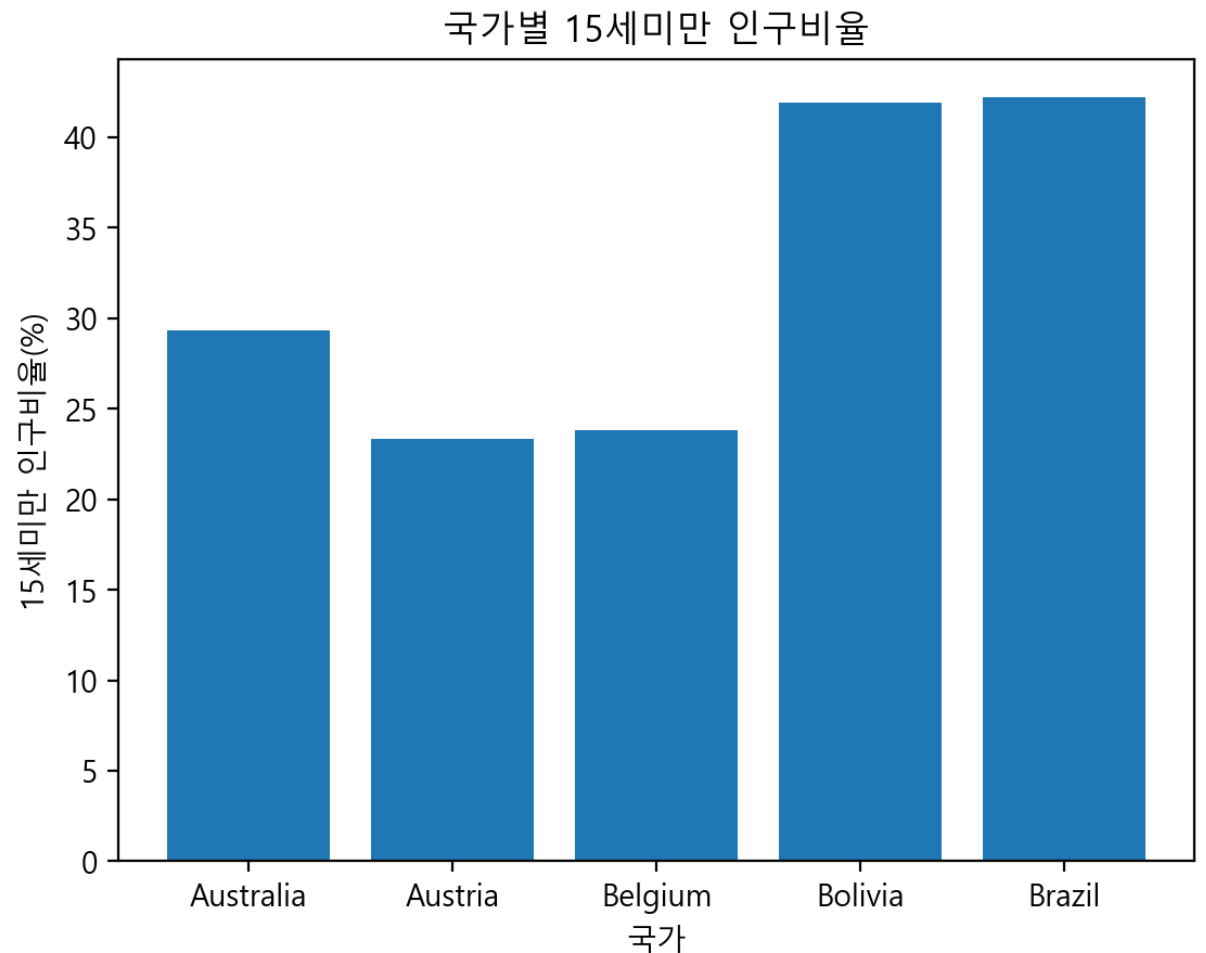
내장 데이터 LifeCycleSavings의 막대 그래프

- “15세 미만의 인구비율”을 나타내는 열 pop15 값을 plt.bar()로 그린 결과

```
plt.bar(df.index, df.pop15)
plt.title("국가별 15세미만 인구비율")
plt.xlabel("국가")
plt.ylabel("15세미만 인구비율(%)");
```

```
df = lcs[:5][['pop15']]
df
```

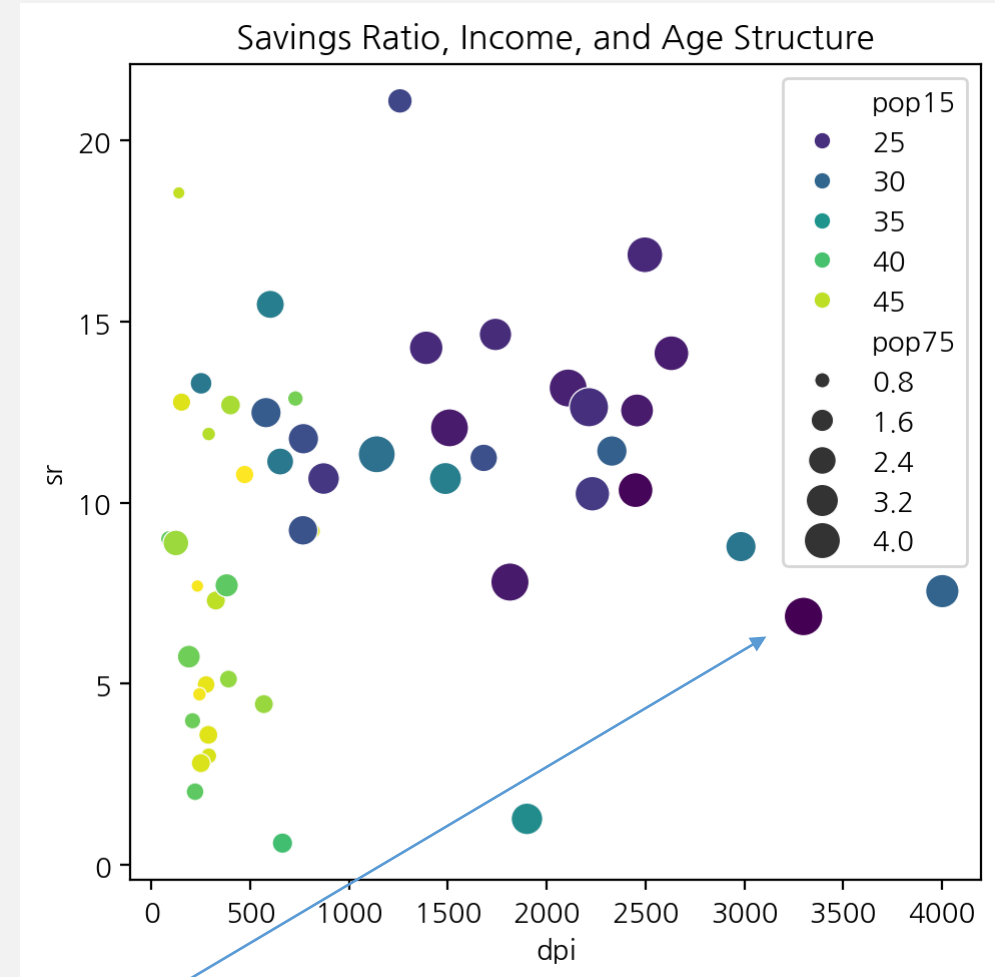
	pop15
Australia	29.35
Austria	23.32
Belgium	23.80
Bolivia	41.89
Brazil	42.19



데이터 시각화 해석 포인트

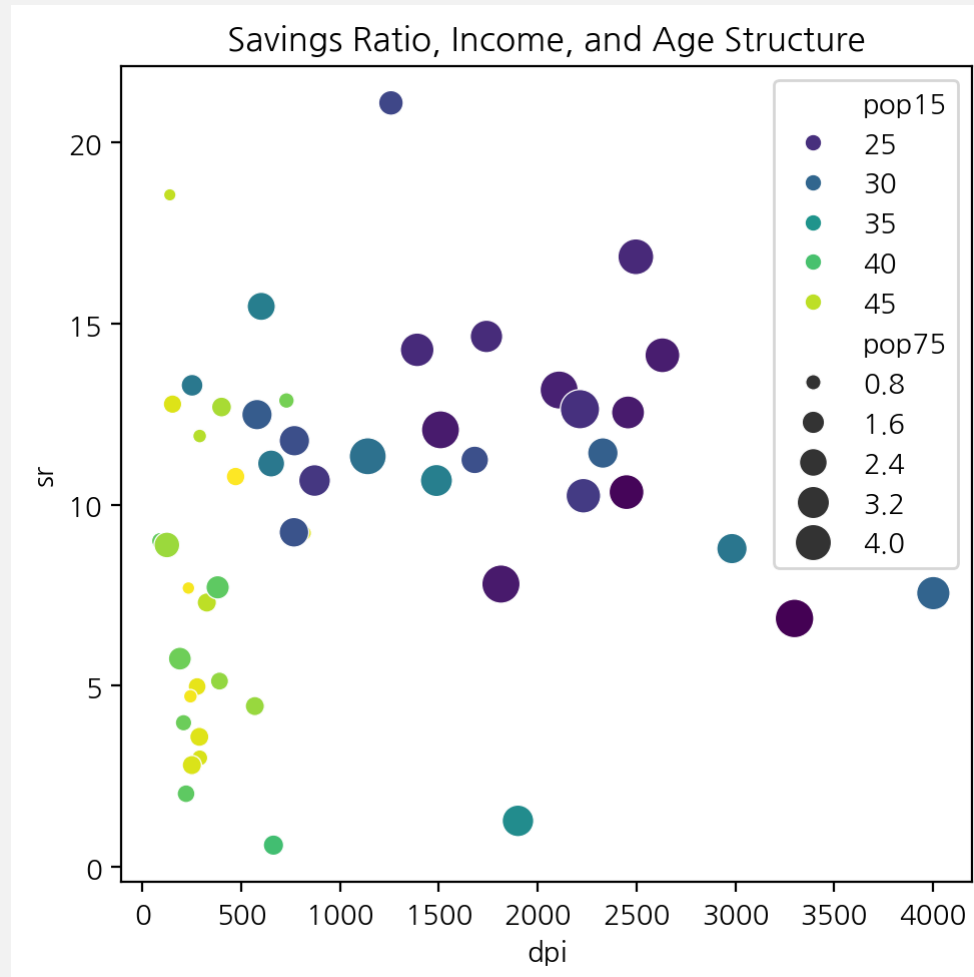
소득이 높을수록 저축률이 높아지는 경향은 있지만, 젊은층과 고령층 비중이 큰 국가는 저축률이 낮아질 수 있음

- **x축(dpi):**
 - 오른쪽으로 갈수록 1인당 가처분소득이 높아짐
- **y축(sr):**
 - 위로 갈수록 저축률이 높아짐
- **점의 색상(pop15): 15세 미만 인구 비율**
 - 밝은 색은 비율이 높음
- **점의 크기(pop75): 75세 초과 인구 비율**
 - 크면 비율이 높음
- **그림에서 읽을 수 있는 의미**
 - 점들이 오른쪽 위로 어느 정도 퍼져 있다면, 소득이 높을수록 저축률도 높아지는 경향
 - 색이 밝은 쪽이 pop15가 높은 국가라면
 - 젊은 인구 비중이 높을수록 저축률이 낮아지는 패턴
- **생애주기 가설 관점**
 - 젊은 인구가 많은 사회는 저축이 줄고,
 - 중장년층 비중이 높고 소득이 높은 사회는 저축이 늘 수 있다는 관점
 - 고소득이라고 항상 저축률이 높은 것은 아님



데이터시각화 코드

```
plt.figure(figsize=(5, 5))
sns.scatterplot(data=lcs, x='dpi', y='sr', hue='pop15',
               palette='viridis', size='pop75', sizes=(20, 200))
plt.title('Savings Ratio, Income, and Age Structure')
plt.xlabel('dpi')
plt.ylabel('sr')
plt.tight_layout()
plt.show()
```



데이터셋 mtcars

```
from pydataset import data
mtc = data('mtcars')
mtc.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 32 entries, Mazda RX4 to Volvo 142E
Data columns (total 11 columns):
#   Column  Non-Null Count  Dtype
---  -
0   mpg     32 non-null     float64
1   cyl     32 non-null     int64
2   disp    32 non-null     float64
3   hp      32 non-null     int64
4   drat    32 non-null     float64
5   wt      32 non-null     float64
6   qsec    32 non-null     float64
7   vs      32 non-null     int64
8   am      32 non-null     int64
9   gear    32 non-null     int64
10  carb    32 non-null     int64
dtypes: float64(5), int64(6)
memory usage: 3.0+ KB
```

메소드 value_counts()를 활용한 막대 그래프

- 두 번째 열 cyl

- 자동차의 실린더(기통) 수
- mtcars에서 실린더 수 분포

- 함수

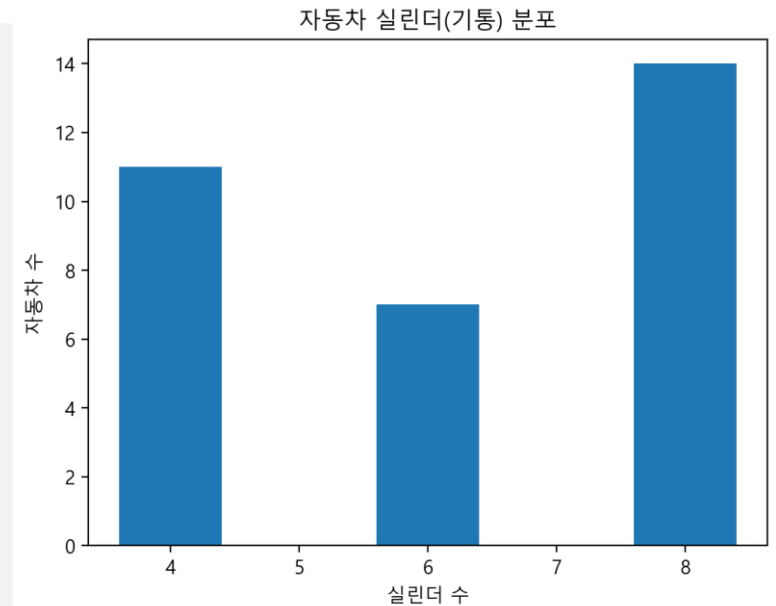
`mtc.cyl.value_counts()`
를 사용

```
mtc_cyl = mtc.cyl.value_counts()  
mtc_cyl
```

✓ 0.0s

```
cyl  
8    14  
4    11  
6     7  
Name: count, dtype: int64
```

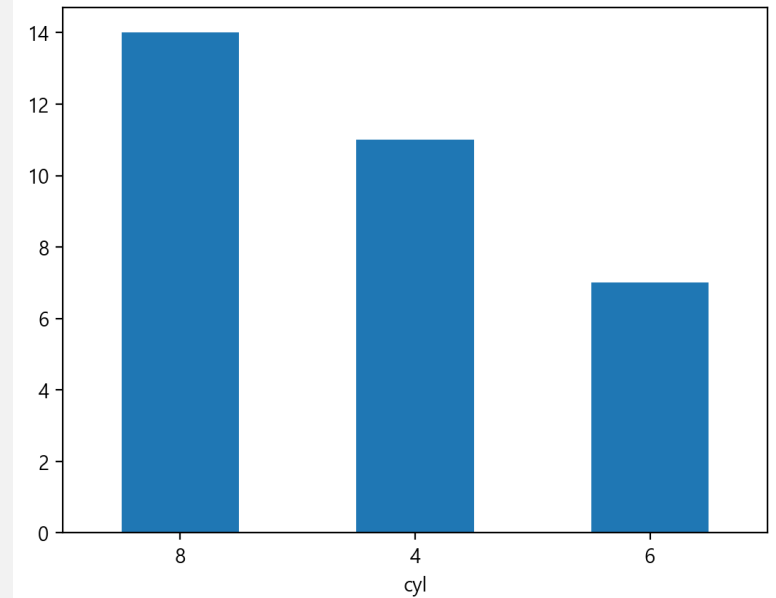
```
plt.bar(mtc_cyl.index, mtc_cyl)  
plt.title("자동차 실린더(기통) 분포")  
plt.xlabel("실린더 수")  
plt.ylabel("자동차 수");
```



- 시리즈에서 그리기

- plot()을 호출
- 이어 메소드 bar() 호출

```
mtc.cyl.value_counts().plot.bar()  
plt.xticks(rotation=360);
```



패키지 seaborn의 함수 sns.countplot()으로 그리는 막대 그래프

sns.countplot()

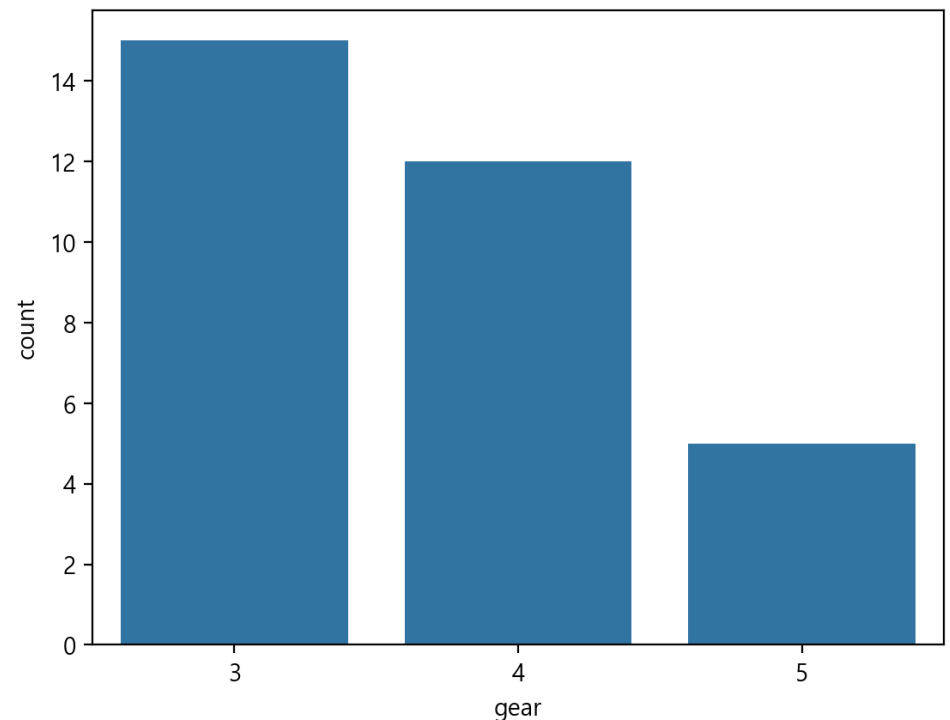
- 데이터 시각화 패키지 seaborn

- 내부적으로 matplotlib을 사용하며 보다 간단하고 다양한 데이터를 시각화할 수 있는 라이브러리

- sns.countplot(mtc, x = 'gear', ...)

- 데이터프레임 mtc의 변수 gear의 빈도 수를 나타내는 막대 그래프 그리기

```
import seaborn as sns  
sns.countplot(mtc, x = 'gear');
```



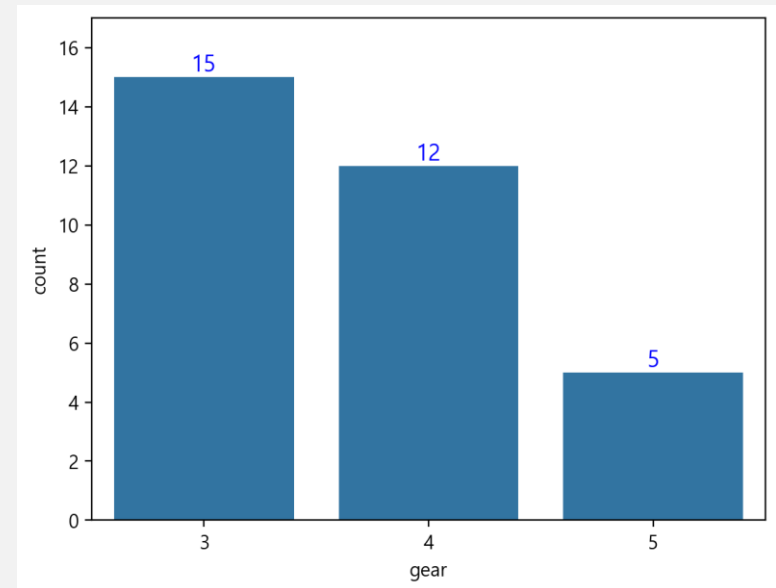
빈도 수 값을 막대 그래프에 표시

- 함수 `sns.countplot()`의 반환 객체는 `Axes`라는 객체
 - 반환 받은 `ax`를 활용
 - `ax.patches`로 그래프의 각 막대(patch)에 대해 반복문을 실행
 - `p.get_height()`로 현재 막대의 높이, 즉 빈도 수를 가져오고
 - `ax.text(x, y, s = int(height), ...)`로 빈도수 값 `int(height)`를 막대 위 위치 `(x, y)`에 표시

```
import seaborn as sns
ax = sns.countplot(data = mtc, x='gear',
                  order = mtc.gear.value_counts().index)

for p in ax.patches:
    height = p.get_height() # 빈도 수
    ax.text(x = (p.get_x() + p.get_width() / 2.), y = (height + .2),
           s = int(height), ha = 'center', size = 12, color = 'blue')

ax.set_ylim(0, 17) # y축의 범위를 0부터 17까지로 설정
plt.show()
```



Axes.text의 인자

`Axes.text(x, y, s, fontdict=None, **kwargs)`

- `ax.text(x = (p.get_x() + p.get_width() / 2.), y = (height + .2), s = int(height), ha = 'center', size = 12, color = 'blue')`:
 - `x = (p.get_x() + p.get_width() / 2.)`: 텍스트를 막대의 중앙에 위치
 - `y = (height + .2)`: 텍스트를 막대의 바로 위에 위치
 - `s = int(height)`: 텍스트 내용은 막대의 높이 (빈도 수)로 설정
 - `ha = 'center'`: 텍스트를 중앙 정렬
 - 위에서 지정한 `x` 좌표가 중앙에 오도록
 - 이것을 지정하지 않으면 왼쪽 정렬이 되어 글씨가 약간 오른쪽으로 표시
 - `size = 12`: 텍스트 크기를 12로 설정
 - `color = 'blue'`: 텍스트 색상을 파란색으로 설정

9.2 상자그림과 히스토그램

-



사분위수

- 변수 `mtc.mpg`의 사분위수
 - 메소드 `quantile([0., .25, .50, .75, 1])`를 사용

-

```
from pydataset import data
mtc = data('mtcars')
mtc.mpg.quantile([0., .25, .50, .75, 1])
```

✓ 0.0s

0.00	10.400
0.25	15.425
0.50	19.200
0.75	22.800
1.00	33.900

Name: mpg, dtype: float64

```
mtc.mpg.describe()[3:]
```

min	10.400
25%	15.425
50%	19.200
75%	22.800
max	33.900

Name: mpg, dtype: float64

상자그림 그리기 함수 boxplot()

- 상자그림(boxplot)

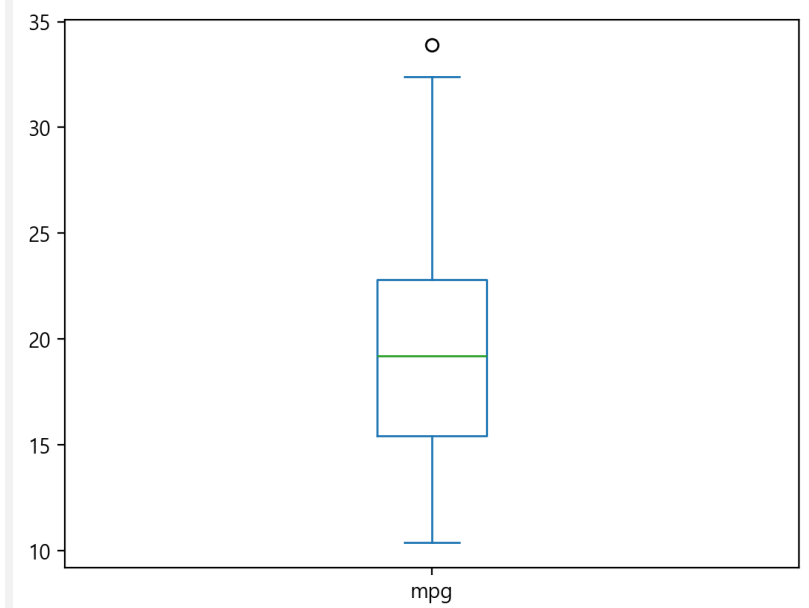
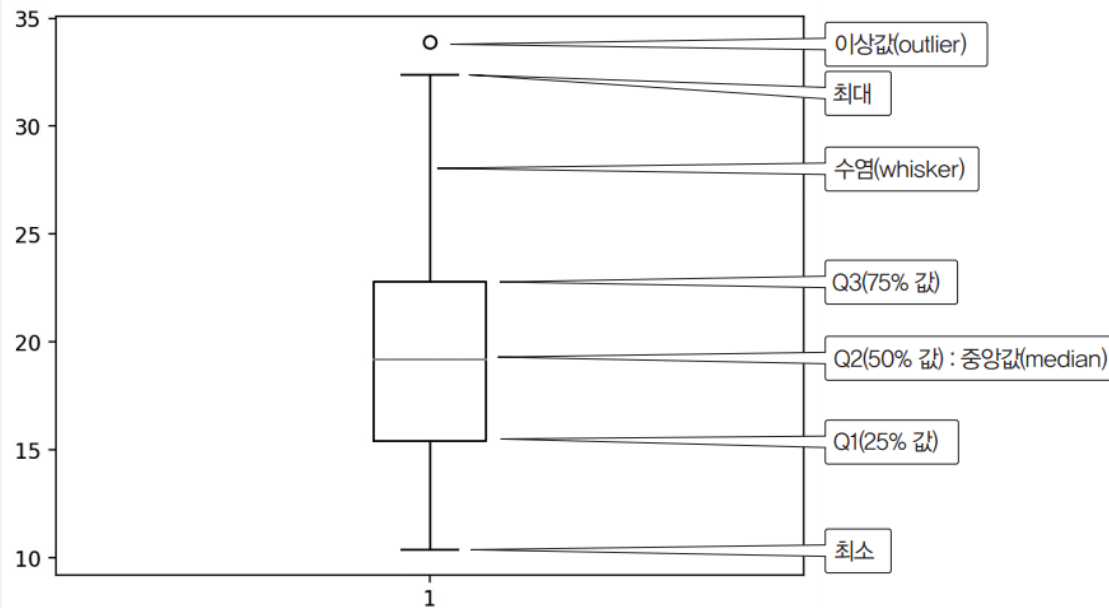
- 데이터의 대략적인 분포와 개별적인 이상치들을 동시에 보여줄 수 있는 차트(chart)
 - 서로 다른 그룹의 분포를 쉽게 비교할 수 있도록 도와주는 시각화 기법으로 가장 널리 쓰이는 시각화 형태
- 상자그림은 수염상자라고도 부름

- boxplot(mtc.mpg)

- 변수 mtc.mpg의 사분위수에서 Q1, Q2, Q3를 상자로 표현하는 상자그림
- 열 mpg에 속한 plot.box()로도 상자그림 가능

```
import matplotlib.pyplot as plt  
  
plt.boxplot(mtc.mpg)  
plt.show()
```

```
mtc.mpg.plot.box();
```

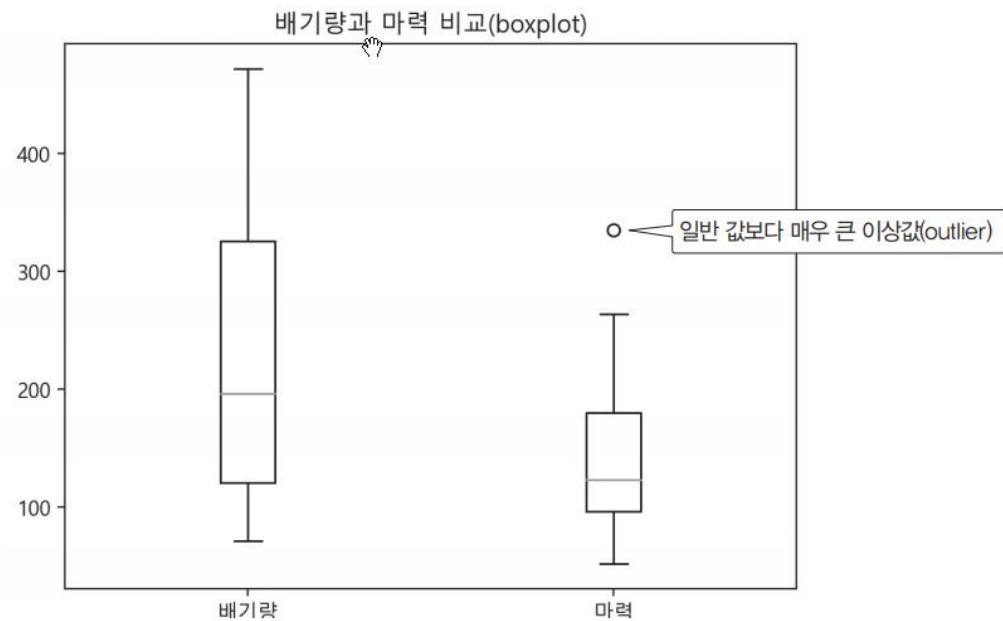


두 개의 boxplot 그리기

- 함수 `boxplot([mtc.disp, mtc.hp])`를 사용
 - 변수 `disp`(배기량)와 `hp`(마력)의 분포를 동시에 알 수 있도록 상자그림을 함께 그릴 수 있음

```
import numpy as np
plt.boxplot([mtc.disp, mtc.hp])
plt.title("배기량과 마력 비교(boxplot)")
plt.xticks(ticks = np.arange(1, 3), labels = ["배기량", "마력"])
plt.show()
```

ticks: 눈금 위치 목록으로 [1, 2]도 가능 (옵션수염(whisker))



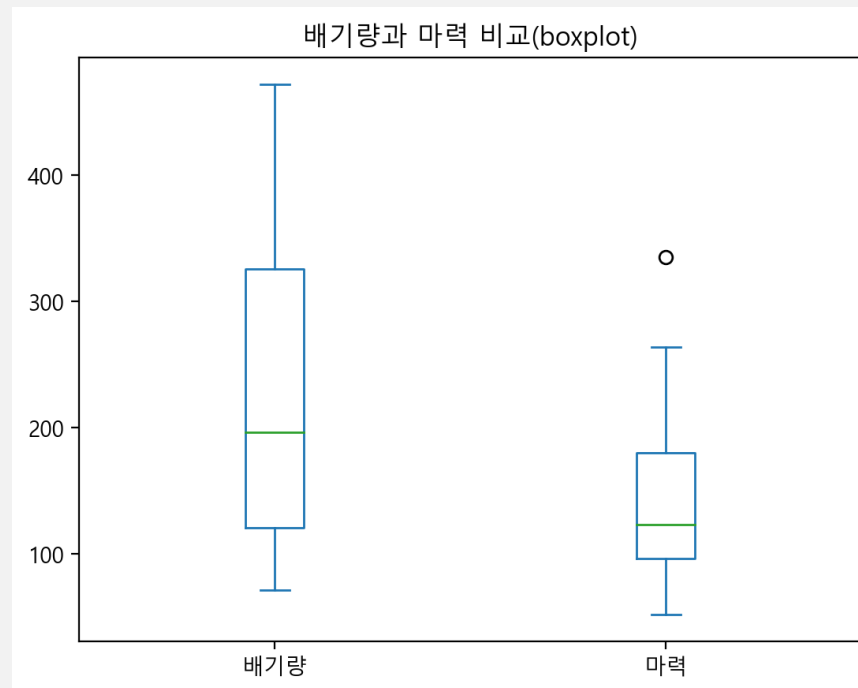
이상값(outlier)

- 이상값

- 상자그림에서 맨 위의 ○ 표시는 이상값(outlier)
- 박스상자의 위나 아래에서 박스상자의 길이의 1.5배 이상 떨어져 있는 매우 크거나 작은 특이한 값
- 데이터 분석에서 이상값의 처리가 매우 중요하므로 최대나 최소 값 밖으로 표시

- 데이터프레임의 메소드 plot()으로도 간단히 그림상자를 그릴 수 있음

```
mtc[['disp', 'hp']].plot(title="배기량과 마력 비교(boxplot)", kind='box')  
plt.xticks(ticks = np.arange(1, 3), labels = ["배기량", "마력"])  
plt.show()
```



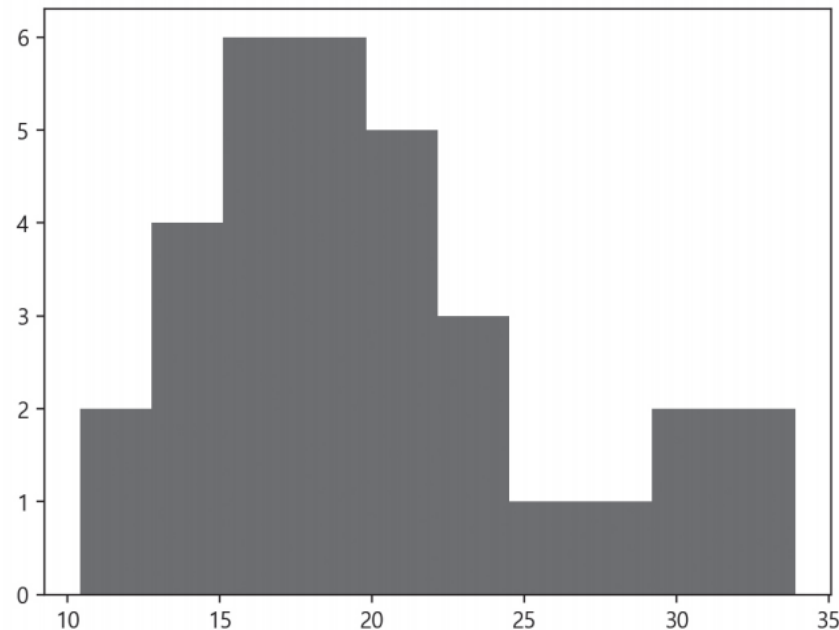
함수 hist()를 활용한 히스토그램

- 자동차 데이터 mtcars에서 연비 데이터 변수인 mtc.mpg로 그린 도수분포표 결과
 - 막대 구간을 지정하지 않으면 자동으로 최소부터 최대까지 10개의 구간을 정해 알아서 그림
 - 결과에서 그림이 그려지기 전에 그림에 사용한 빈도수와 구간이 표시

```
plt.hist(mtc.mpg)
(array([2., 4., 6., 6., 5., 3., 1., 1., 2., 2.]),
 array([10.4 , 12.75, 15.1 , 17.45, 19.8 , 22.15, 24.5 , 26.85, 29.2 ,
        31.55, 33.9 ]),
<BarContainer object of 10 artists>)
```

10개의 구간의 빈도수이다.

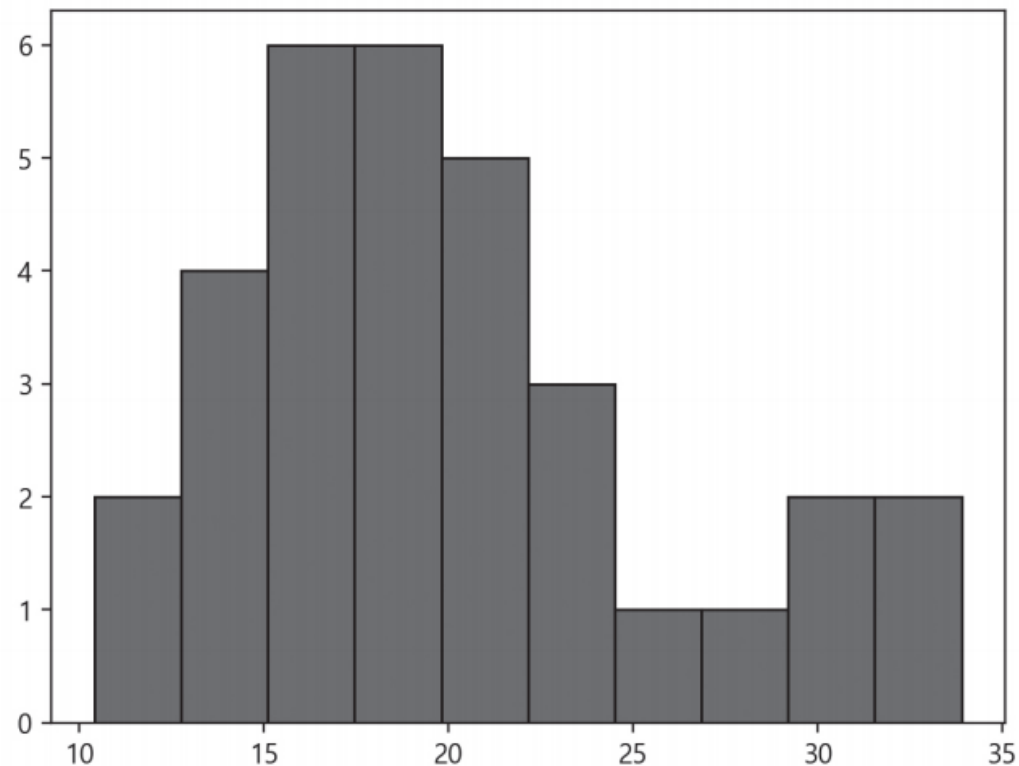
10개의 구간을 나눈 경계 값이다.



함수 hist(..., edgecolor='black')

- 함수 hist()에서 인자 edgecolor에 색상을 지정하면 구간 구분이 명확
 - 문장 마지막에 세미콜론(;)을 넣으면 출력 글자는 표시되지 않음

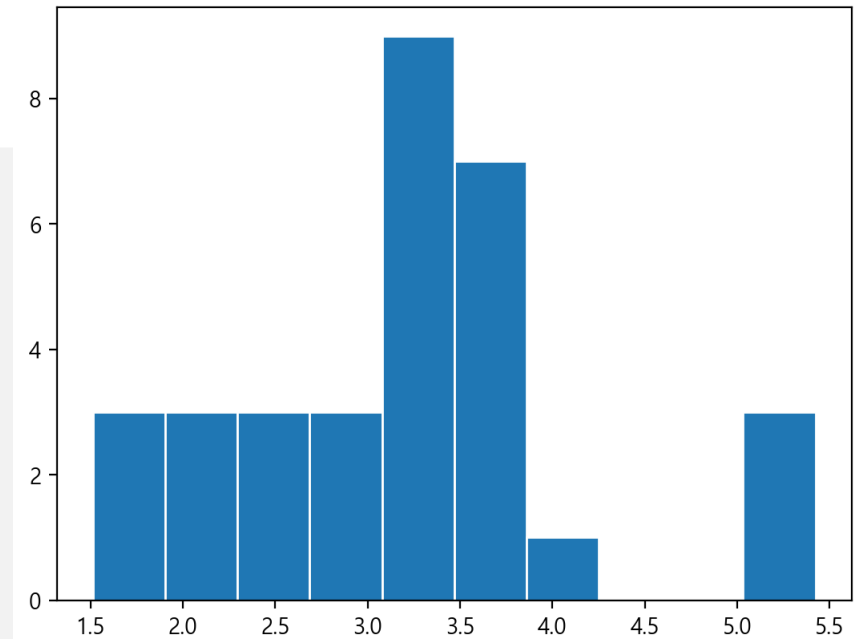
```
plt.hist(mtc.mpg, edgecolor='black');
```



자동차 무게 변수인 mtc.wt로 그린 dots분포표

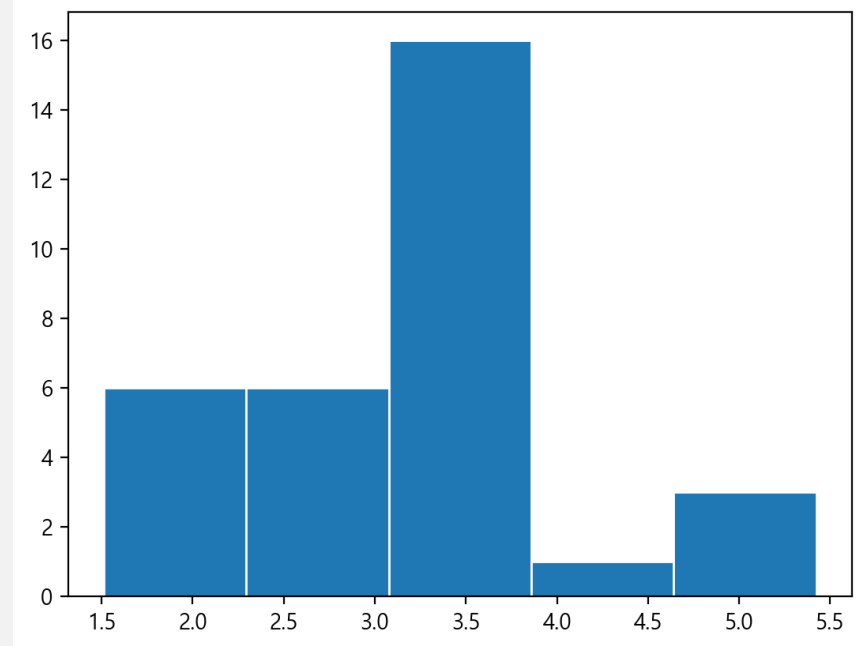
- 인자 bins= 없이

```
plt.hist(mtc.wt, edgecolor='white')
✓ 0.1s
(array([3., 3., 3., 3., 9., 7., 1., 0., 0., 3.]),
 array([1.513, 1.9041, 2.2952, 2.6863, 3.0774, 3.4685, 3.8596, 4.2507,
        4.6418, 5.0329, 5.424 ]),
 <BarContainer object of 10 artists>)
```



- 인자 bins=5
- 구간이 5개

```
plt.hist(mtc.wt, bins = 5, edgecolor='white')
(array([ 6.,  6., 16.,  1.,  3.]),
 array([1.513, 2.2952, 3.0774, 3.8596, 4.6418, 5.424 ]),
 <BarContainer object of 5 artists>)
```



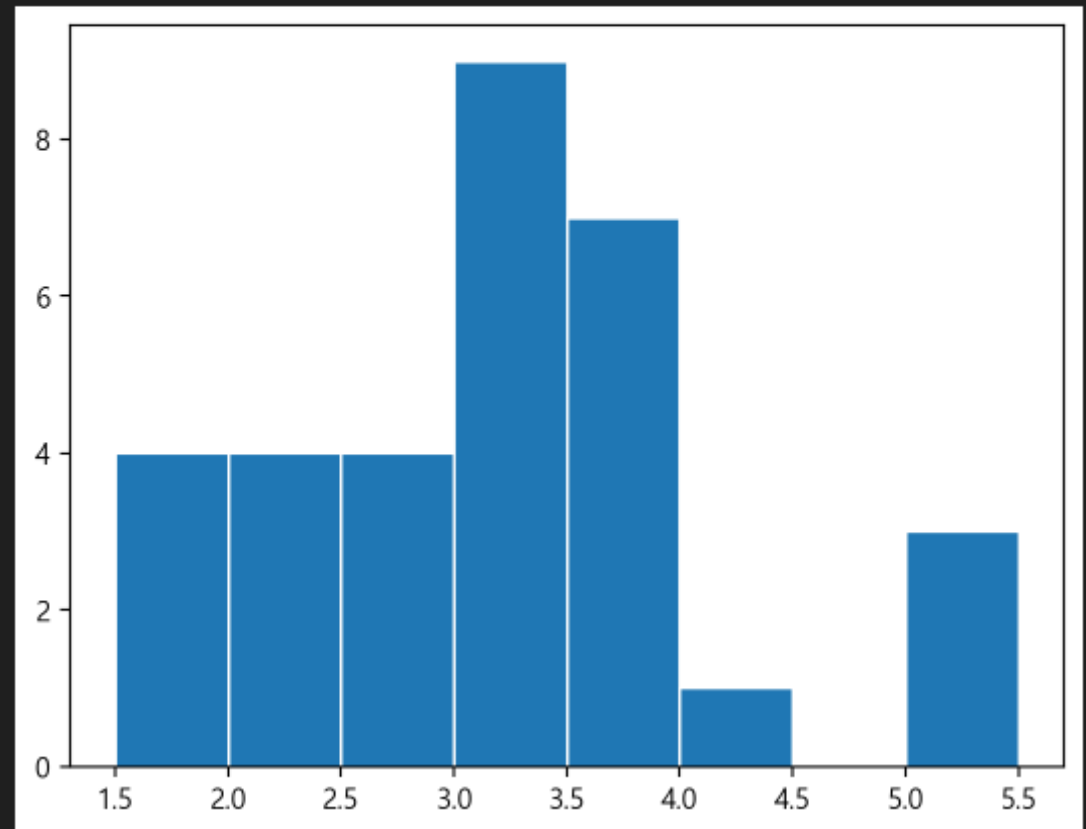
`plt.hist(..., bins = np.arange(1.5, 5.6, 0.5), ...)`

구간을 직접 등분하는 방법으로 지정하는 방법

- 인자 구간 `bins = np.arange(1.5, 5.6, 0.5)`

- 등분하는 구간을 직접 지정
- 1.5에서 시작하여 5.6보다 작은 값까지 0.5씩 증가하는 숫자 배열을 생성
- bins 배열
 - `array([1.5, 2. , 2.5, 3. , 3.5, 4. , 4.5, 5. , 5.5])`

```
plt.hist(mtc.wt, bins = np.arange(1.5, 5.6, 0.5), edgecolor='white')
✓ 0.2s
(array([4., 4., 4., 9., 7., 1., 0., 3.]),
 array([1.5, 2. , 2.5, 3. , 3.5, 4. , 4.5, 5. , 5.5])),
<BarContainer object of 8 artists>)
```



`plt.hist(mtc.wt, bins = [1.5, 2.5, 4.5, 5.5], density=True)`

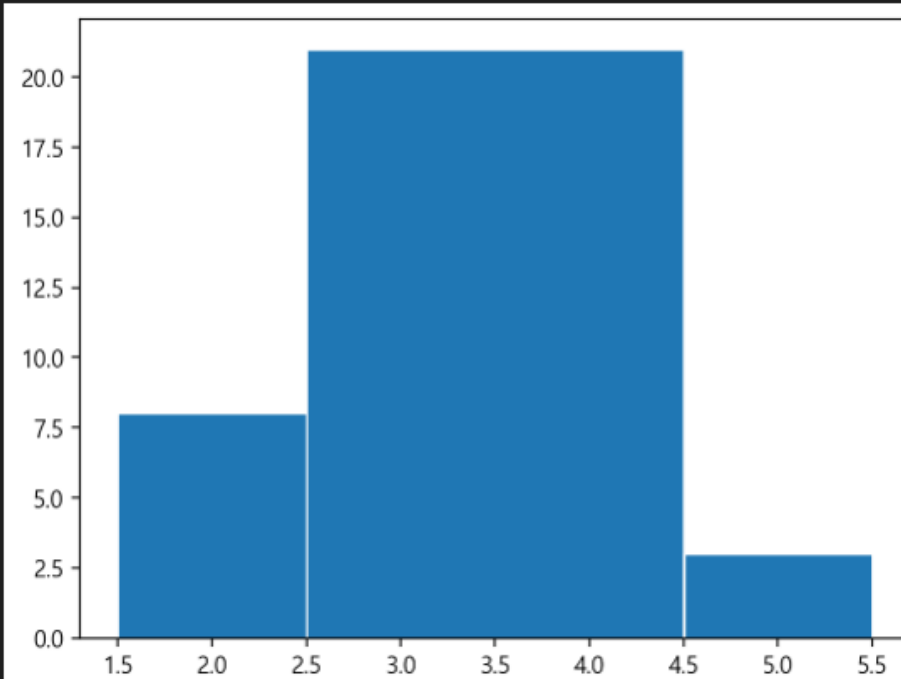
`bins = [1.5, 2.5, 4.5, 5.5]`, 간격이 일정하지 않게도 지정 가능

- 인자 `density=True`를 사용하면 확률 밀도로 표시
 - 막대의 길이가 확률이 되며 막대의 모든 면적을 합하면 1

```
plt.hist(mtc.wt, bins = [1.5, 2.5, 4.5, 5.5], edgecolor='white')
```

✓ 0.1s

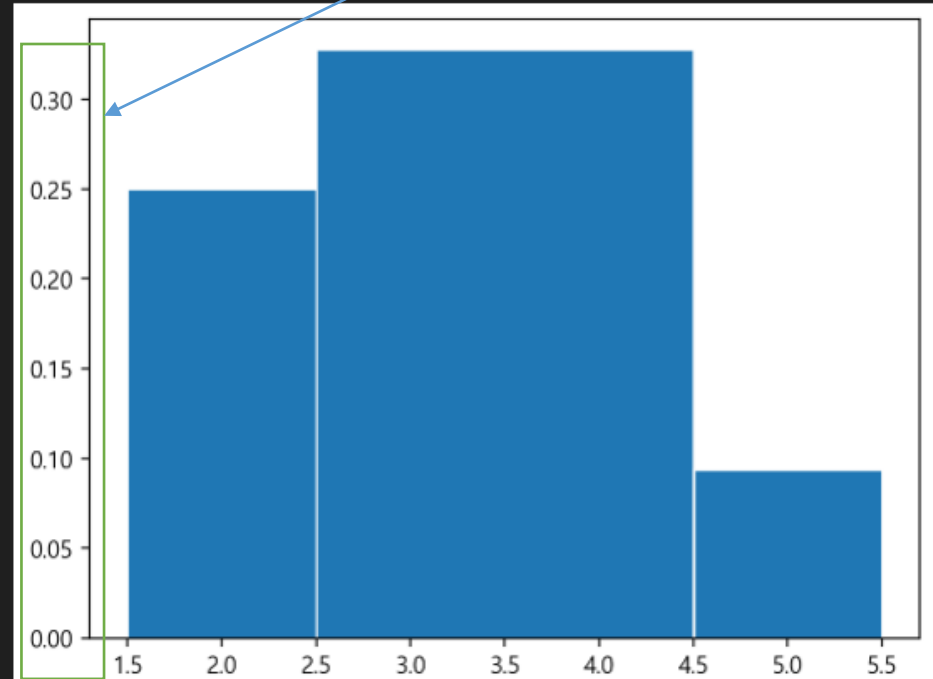
```
(array([ 8., 21.,  3.]),  
array([1.5, 2.5, 4.5, 5.5]),  
<BarContainer object of 3 artists>)
```



```
plt.hist(mtc.wt, bins = [1.5, 2.5, 4.5, 5.5], density=True, edgecolor='white')
```

✓ 0.1s

```
(array([0.25      , 0.328125, 0.09375 ]),  
array([1.5, 2.5, 4.5, 5.5]),  
<BarContainer object of 3 artists>)
```



유명한 붓꽃 데이터 iris

붓꽃 데이터 iris는 5개의 변수와 150개의 관측 값으로 구성된 데이터프레임

- `iris.Species.unique()`

- 변수 Species

- 문자열 개체(object)로 표시되는 범주형
 - 붓꽃의 품종인 `setosa`, `versicolor`, `virginica` 중의 한 값

열 이름	타입	설명	단위
sepal_length	float	꽃받침 길이	cm
sepal_width	float	꽃받침 너비	cm
petal_length	float	꽃잎 길이	cm
petal_width	float	꽃잎 너비	cm
species	object	붓꽃 종류 (setosa/versicolor/virginica)	-

```
from pydataset import data
iris = data('iris')
iris.info()
```

✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>
Index: 150 entries, 1 to 150
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Sepal.Length    150 non-null   float64
1   Sepal.Width     150 non-null   float64
2   Petal.Length    150 non-null   float64
3   Petal.Width     150 non-null   float64
4   Species         150 non-null   object
dtypes: float64(4), object(1)
memory usage: 7.0+ KB
```

```
iris.Species.unique()
```

✓ 0.0s

```
array(['setosa', 'versicolor', 'virginica'], dtype=object)
```

함수 hist()의 반환 값 처리

- 함수 hist()가 반환하는 3개의 반환 값
 - 각각 n, bins, patches에 대입해 출력
 - n
 - 구간의 빈도수
 - bins
 - 구간의 경계 값 배열
 - patches
 - 막대컨테이너(barContainer) 객체

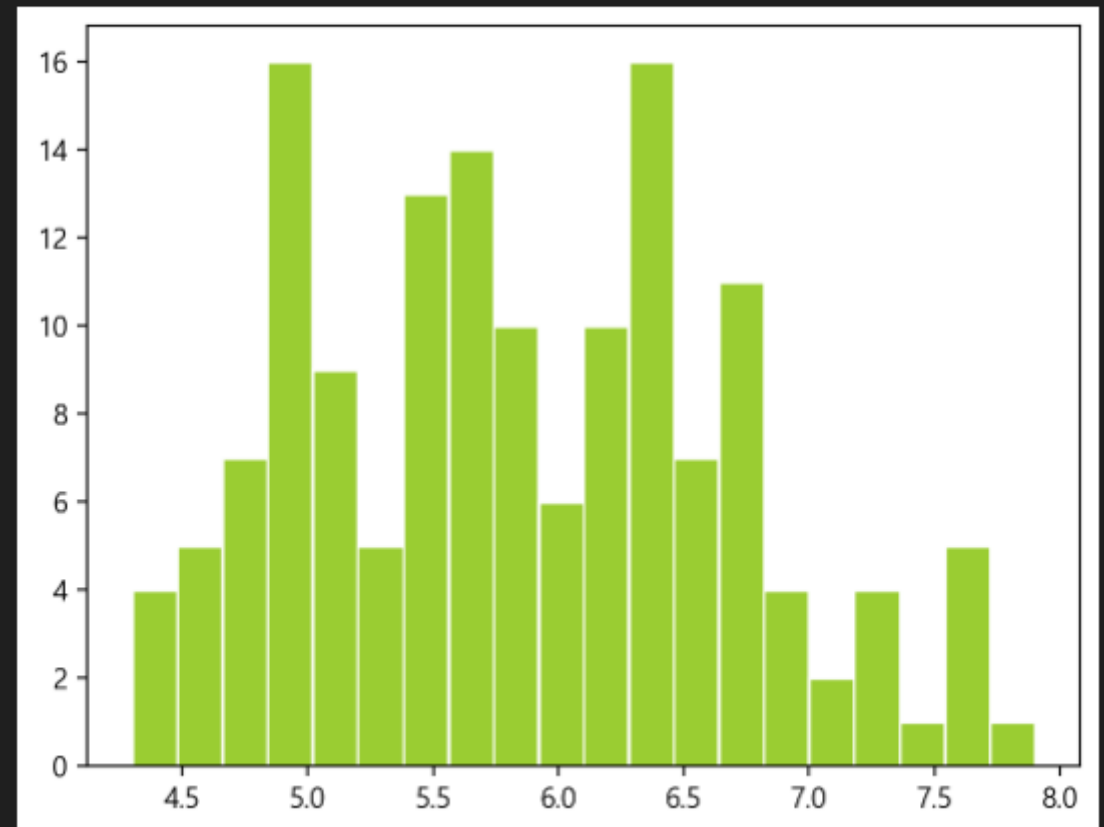
```
n, bins, patches = plt.hist(iris['Sepal.Length'], bins = 20,  
                             color='yellowgreen', edgecolor='white')  
print(n)  
print(bins)  
print(patches)
```

✓ 0.1s

```
[ 4.  5.  7. 16.  9.  5. 13. 14. 10.  6. 10. 16.  7. 11.  4.  2.  4.  1.  
 5.  1.]
```

```
[4.3  4.48 4.66 4.84 5.02 5.2   5.38 5.56 5.74 5.92 6.1   6.28 6.46 6.64  
 6.82 7.   7.18 7.36 7.54 7.72 7.9 ]
```

```
<BarContainer object of 20 artists>
```



값에 따라 막대의 색상을 수정

변수 patches에서 patches[i].set_facecolor()

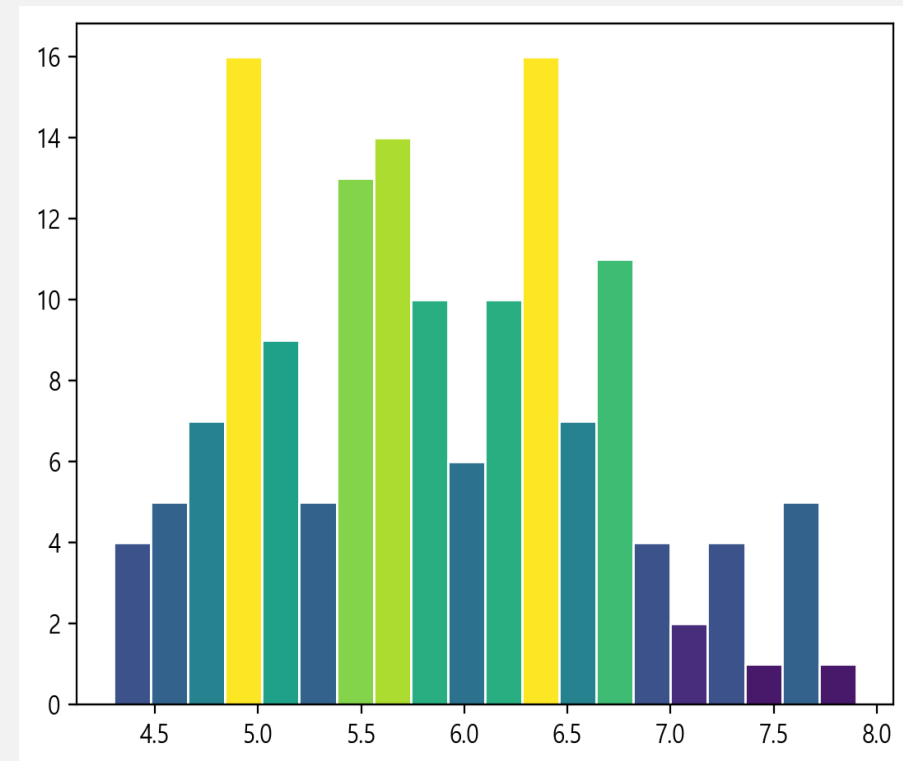
- 막대마다 서로 다른 색을 지정

- 막대컨테이너(barContainer) 객체인 변수 patches
 - patches[i].set_facecolor()로 각 막대의 색상을 지정

- 색상을 plt.cm.viridis(n[i]/max(n))으로 지정

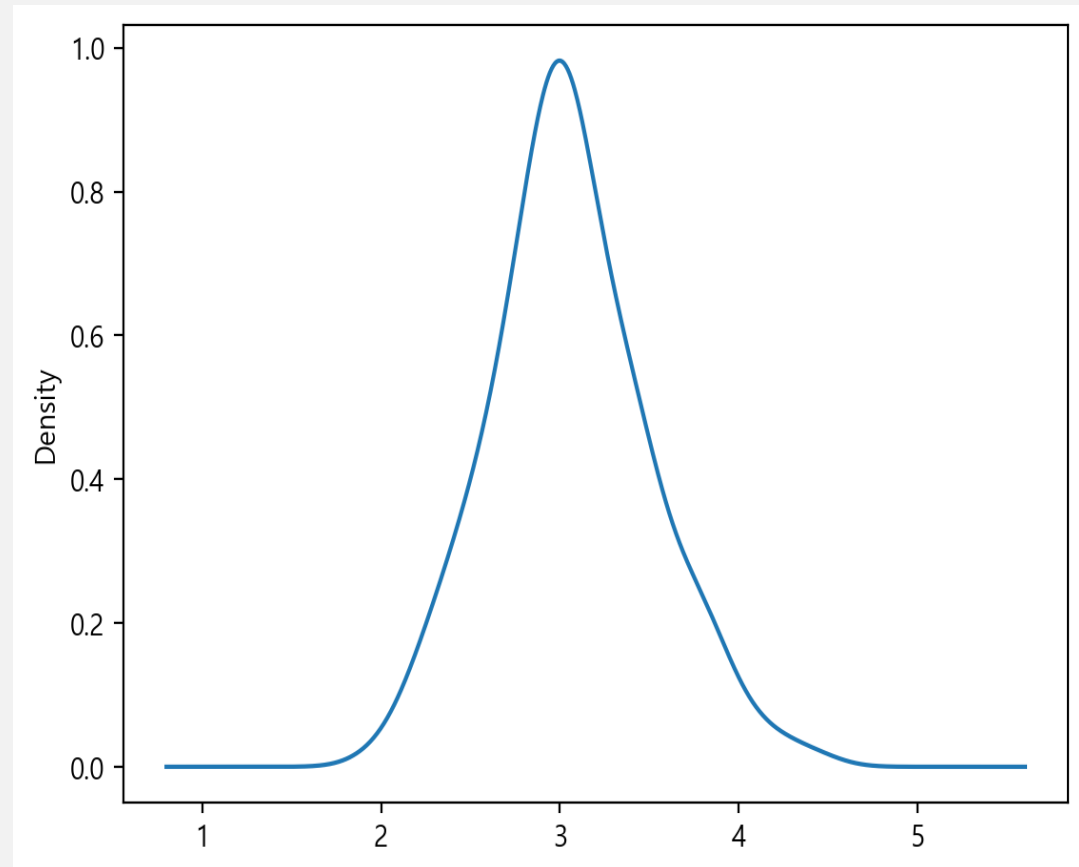
- 여러 색상을 갖고 있는 색상맵 plt.cm.viridis
- 값 빈도수 n[i]에 따라 n[i]/max(n)로 다른 색상을 반환
- 빈도 수가 작은 값은 보라색으로 큰 값은 노란색

```
n, bins, patches = plt.hist(iris['Sepal.Length'],  
                             bins = 20,  
                             color='yellowgreen',  
                             edgecolor='white')  
  
for i in range(len(patches)):  
    patches[i].set_facecolor(plt.cm.viridis(n[i]/max(n)))  
  
plt.show()
```



커널 밀도 추정(kernel density estimation) 선 그리기

- `iris['Sepal.Width'].plot.density()`
- `iris['Sepal.Width'].plot.kde()`
 - 그려진 하부 면적을 모두 합하면 1



패키지 seaborn의 함수 sns.histplot()과 sns.rugplot()

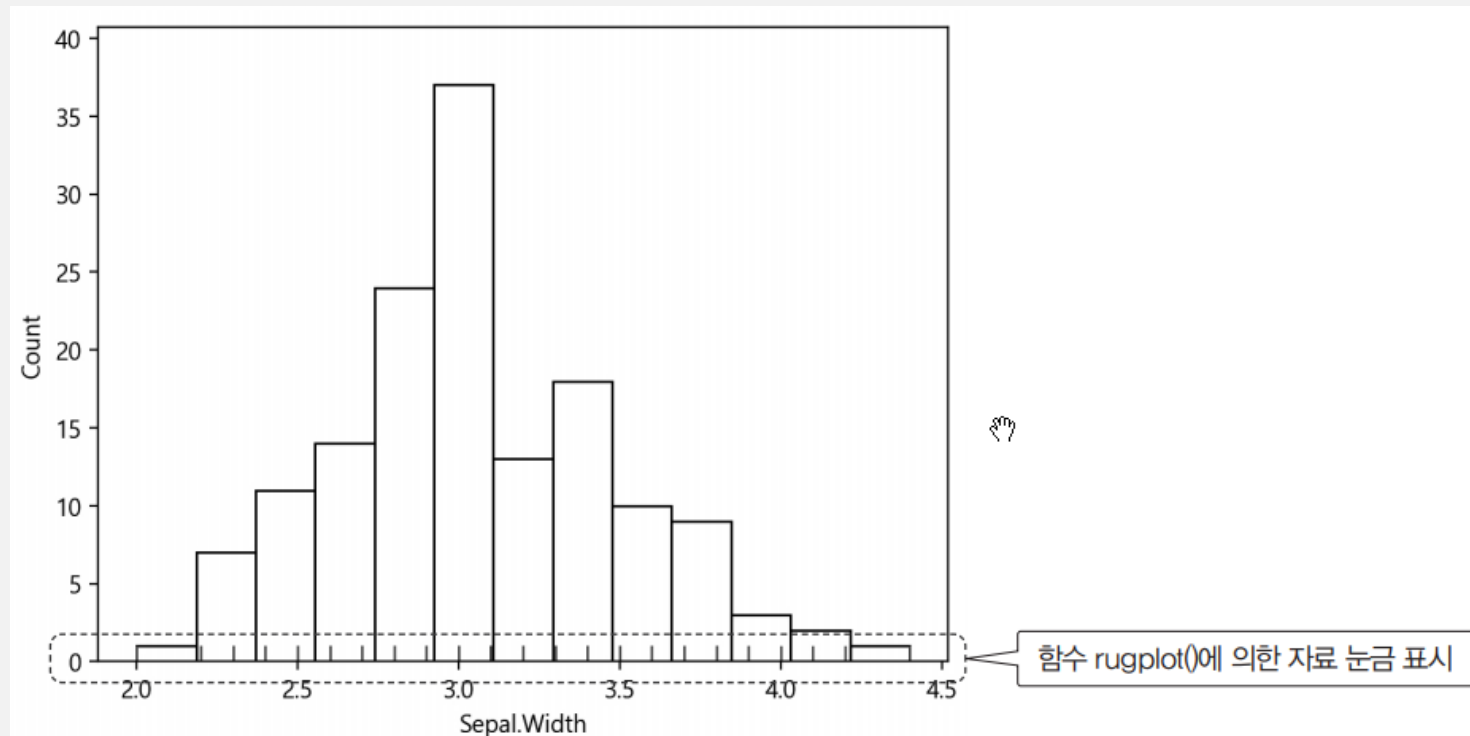
- **sns.histplot()**

- 꽃받침 너비 변수
 - iris['Sepal.Width']
의 도수분포표

```
import seaborn as sns
sns.histplot(data=iris, x = iris['Sepal.Width'], color='white')
sns.rugplot(data=iris, x = iris['Sepal.Width']);
```

- **sns.rugplot()**

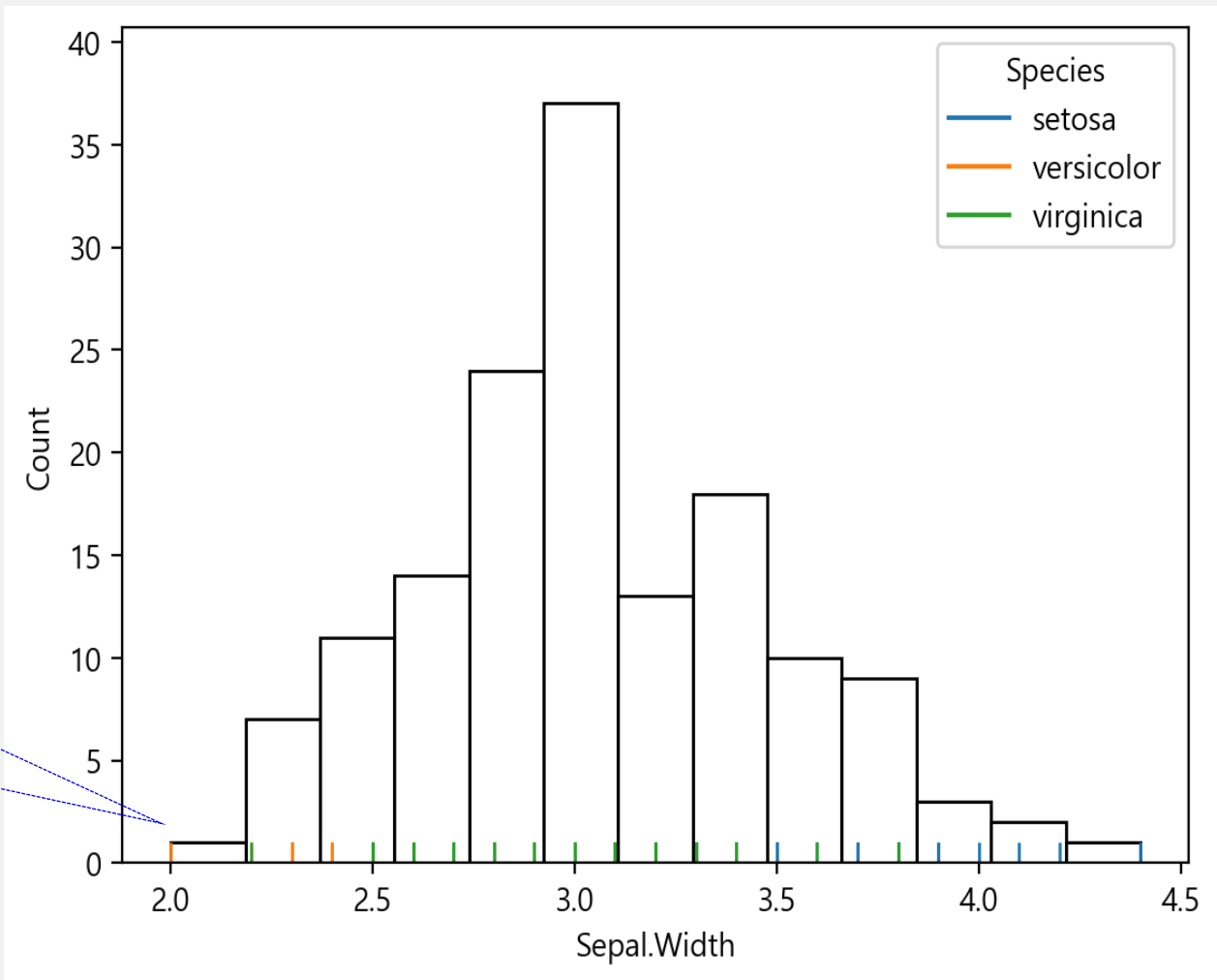
- x 축에 해당 자료의 위치를 눈금으로 표시



함수 `sns.rugplot()` 인자 `hue='Species'`

- 붓꽃의 품종에 따라 색상이 다른 눈금

```
sns.rugplot(data = iris, x = iris['Sepal.Width'],  
            height=0.05, hue='Species');
```

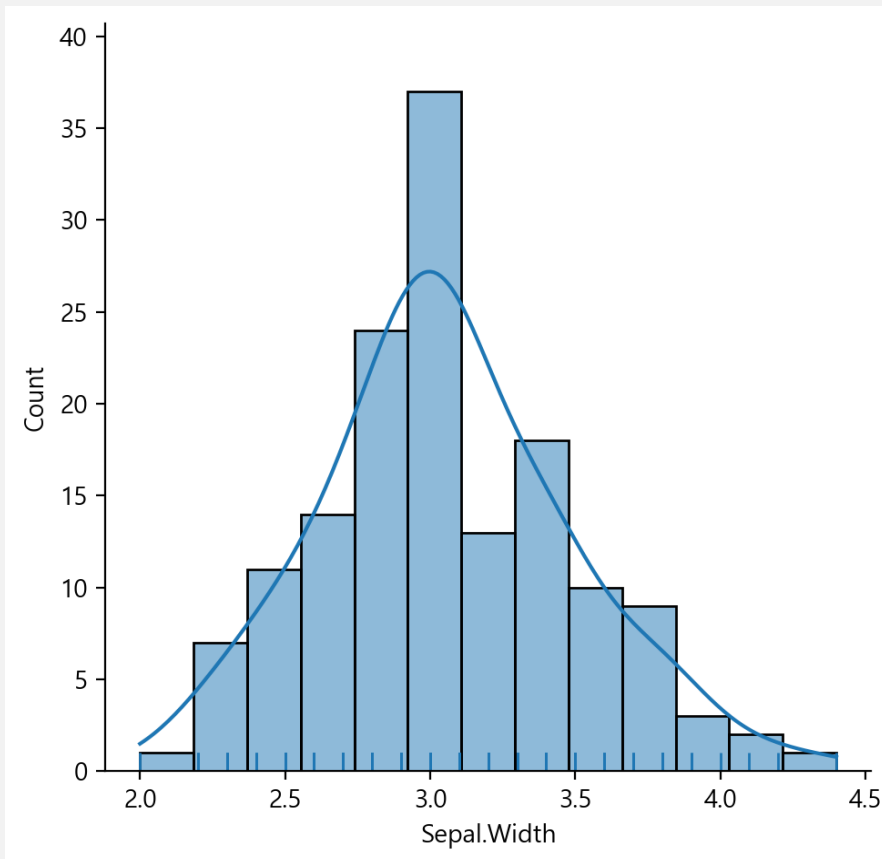


품종에 따라 자료
눈금 표시가 다르다.

```
sns.displot(data = iris, x = iris['Sepal.Width'], kde=True, rug=True);
```

- 도수분포표와 자료 값의 위치인 러그(rug)와 함께 커널 밀도 추정 그래프
 - 함수 sns.displot()에 인자 kde=True, rug=True 를 사용

```
import seaborn as sns  
sns.displot(data = iris, x = iris['Sepal.Width'], kde=True, rug=True);
```



`sns.displot(data = iris, x = iris['Sepal.Width'], col="Species", ...);`

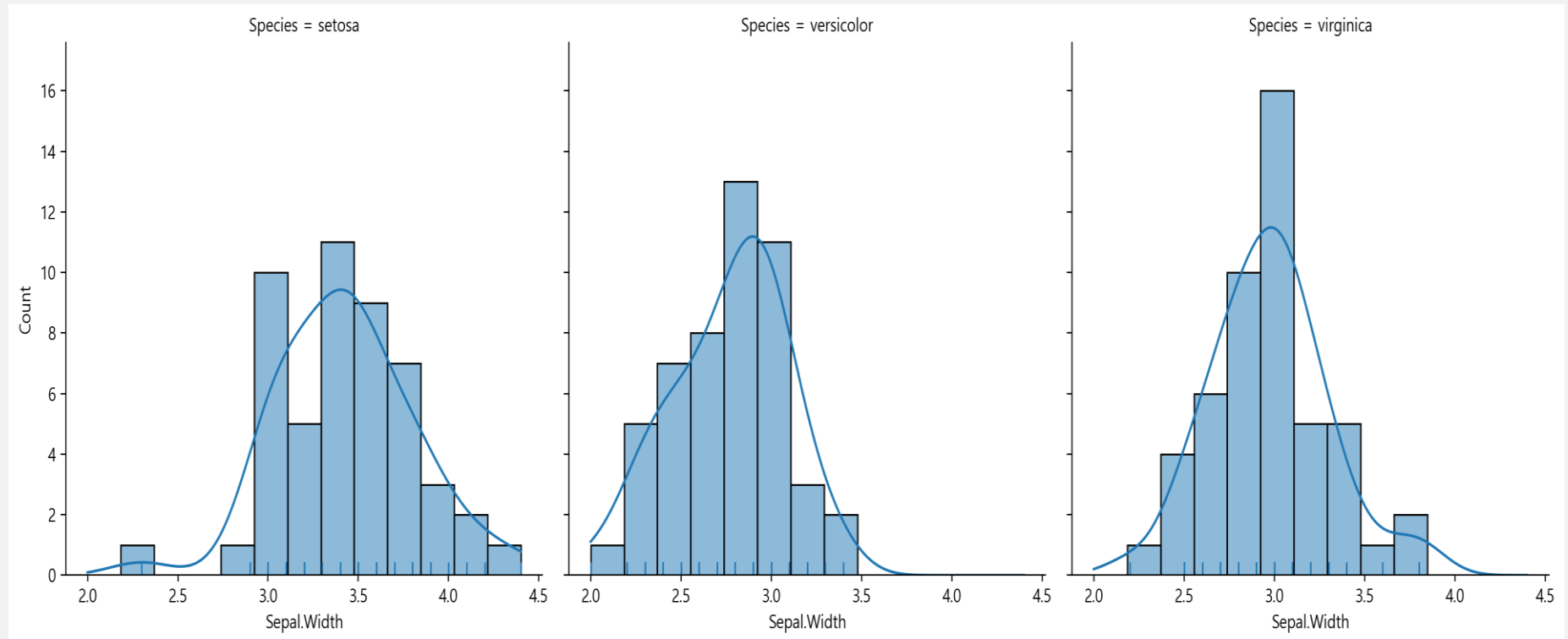
인자 col에 지정한 종류 수대로 열(columns)에 그리기

- 인자 col="Species"의 사용

- 붓꽃의 품종에 따라 가로로 세 개의 도수분포표

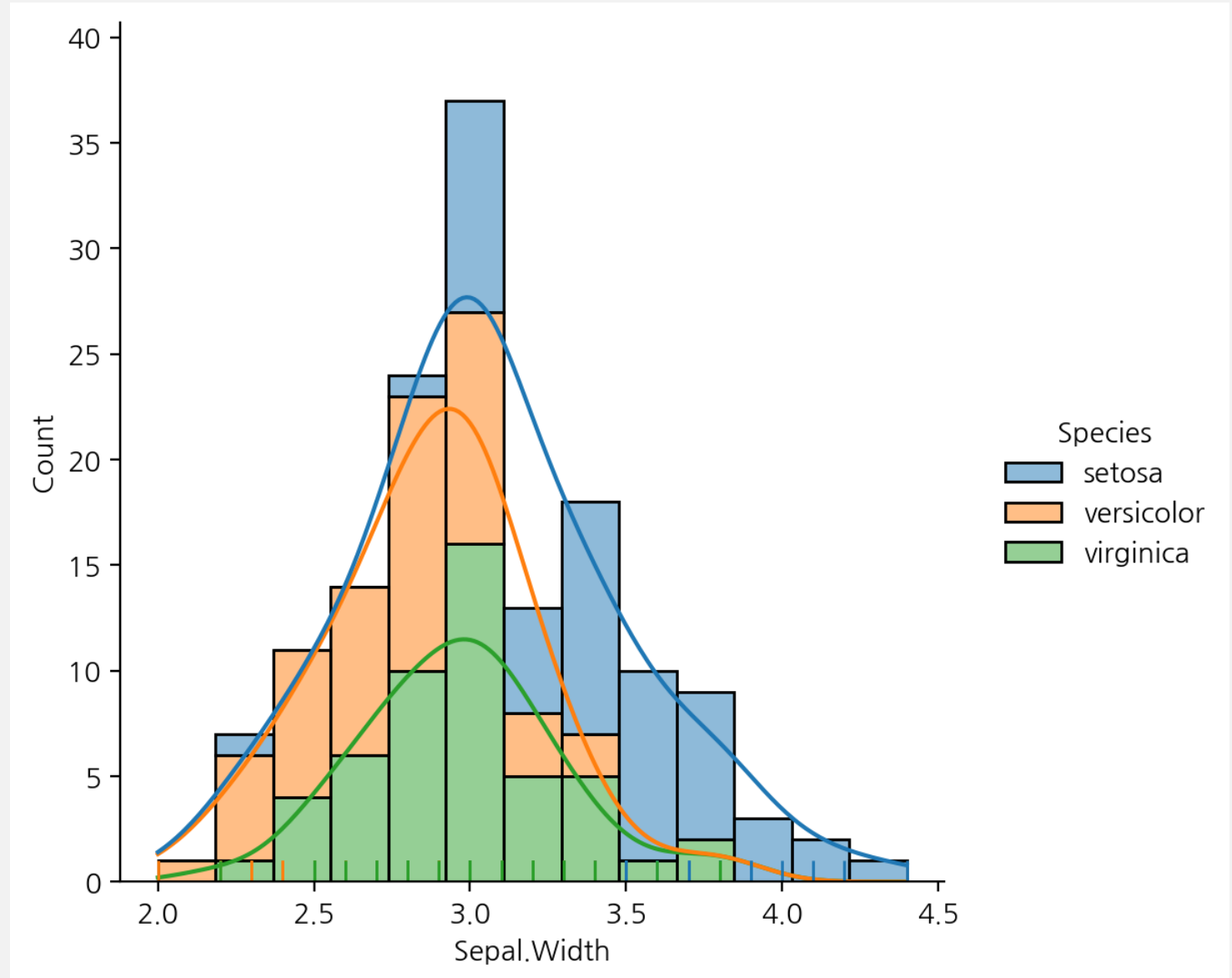
```
import seaborn as sns
```

```
sns.displot(data = iris, x = iris['Sepal.Width'], col="Species",  
            kde=True, rug=True);
```



```
sns.displot(data = iris, x = iris['Sepal.Width'], kde=True, rug=True,  
            multiple="stack", hue="Species");
```

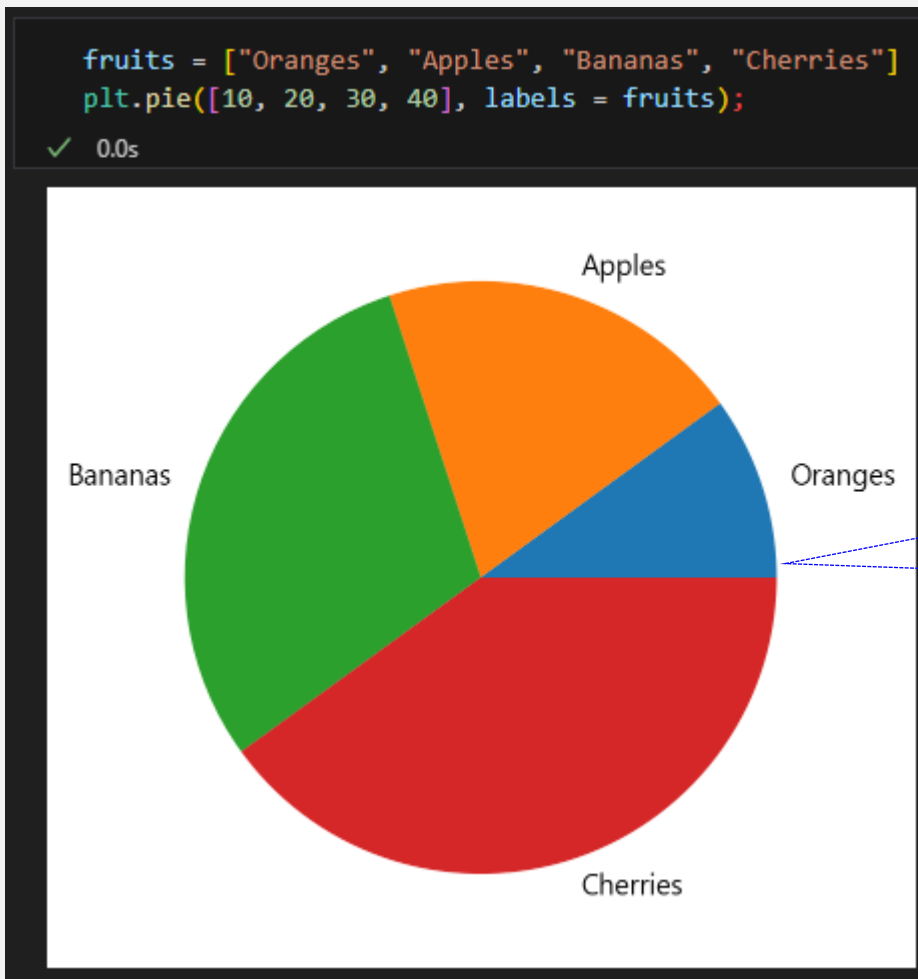
- 겹치지 않게 하려면
- multiple="stack"



함수 pie()로 활용한 파이 차트

원형의 파이 조각을 나눈 것처럼 데이터의 상대 비율을 그리는 원형 그래프

- 데이터 [10,20,30,40]를 각각의 레이블 fruits로 붙인 파이 차트
 - plt.pie([10,20,30,40], labels = fruits)



첫 파이 조각이 그려지는 시작 부분이며
시계 반대 방향으로 그려진다.

데이터 mtc의 변수 cyl(실린더 수)의 파이 차트

- 인자 `autopct='%0.1f%%'`
 - 백분율로 비율을 표시

```
plt.pie(mtc.cyl.value_counts().sort_index(),
        colors = ["violet", "pink", "aqua"],
        autopct='%0.1f%%', labels = txt)

plt.title("기통 수에 따른 자동차 비율") ;
```

```
mtc.cyl.value_counts().sort_index()
```

```
✓ 0.0s
```

```
cyl
```

```
4    11
```

```
6     7
```

```
8    14
```

```
Name: count, dtype: int64
```

```
mtc.cyl.value_counts().sort_index().index
```

```
✓ 0.0s
```

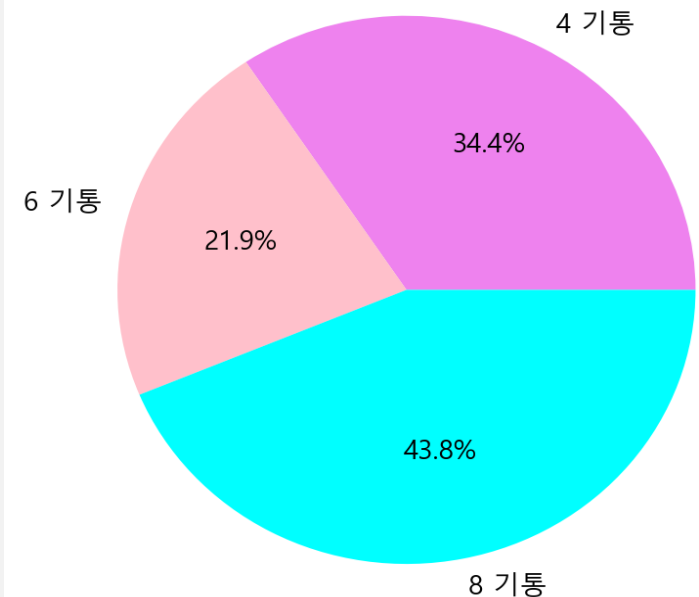
```
Index([4, 6, 8], dtype='int64', name='cyl')
```

```
txt = [str(i) + ' 기통' for i in mtc.cyl.value_counts().sort_index().index]
txt
```

```
✓ 0.0s
```

```
['4 기통', '6 기통', '8 기통']
```

기통 수에 따른 자동차 비율



내장 데이터 mtcars의 변수 gear의 빈도 수

```
mtc.gear.value_counts().sort_index()
```

✓ 0.0s

gear

3 15

4 12

5 5

Name: count, dtype: int64

```
labels = [str(i) + ' 기어' for i in mtc.gear.value_counts().sort_index().index]  
labels
```

✓ 0.0s

['3 기어', '4 기어', '5 기어']

```
## 라벨의 개수만큼 색상 리스트 생성  
colors = sns.color_palette('Accent', len(labels))  
colors
```

✓ 0.0s



sns.color_palette('Dark2', 10)

```
import seaborn as sns
## 라벨의 개수만큼 색상 리스트 생성
colors = sns.color_palette('Accent', len(labels))
colors
```



```
## 라벨의 개수만큼 색상 리스트 생성
colors = sns.color_palette('Set1', len(labels))
colors
```



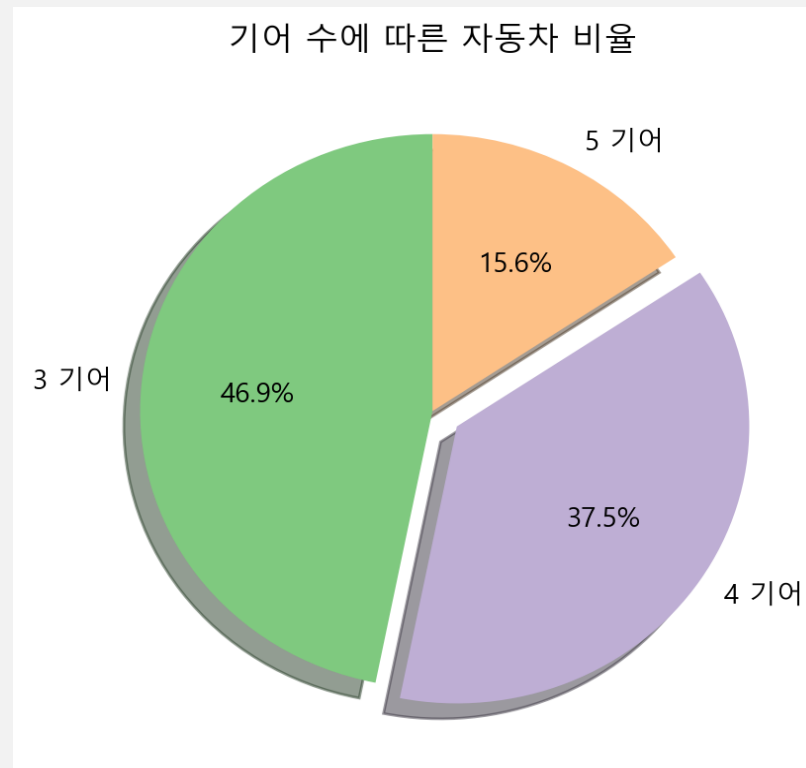
```
## 라벨의 개수만큼 색상 리스트 생성
colors = sns.color_palette('Set2', 7)
colors
```



파이 차트 인자 shadow startangle

- 그림자 효과를 추가하는 shadow, 시작 파이의 위치를 지정하는 startangle

```
colors = sns.color_palette('Accent', len(txt)) ## 라벨의 개수 만큼 색상 리스트 생성
plt.pie(mtc.gear.value_counts().sort_index(), colors=colors, explode=[0, .1, 0],
        shadow = True, startangle = 90, autopct='%0.1f%%', labels = labels)
plt.title("기어 수에 따른 자동차 비율") ;
```



9.3 바이올린과 상관관계 시각화

-



바이올린 개요

- 바이올린 플롯(violin plot)

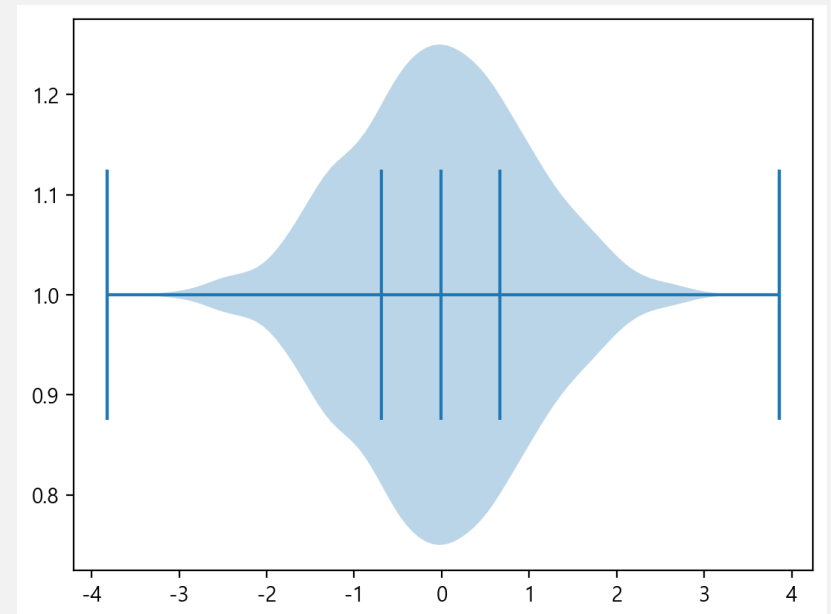
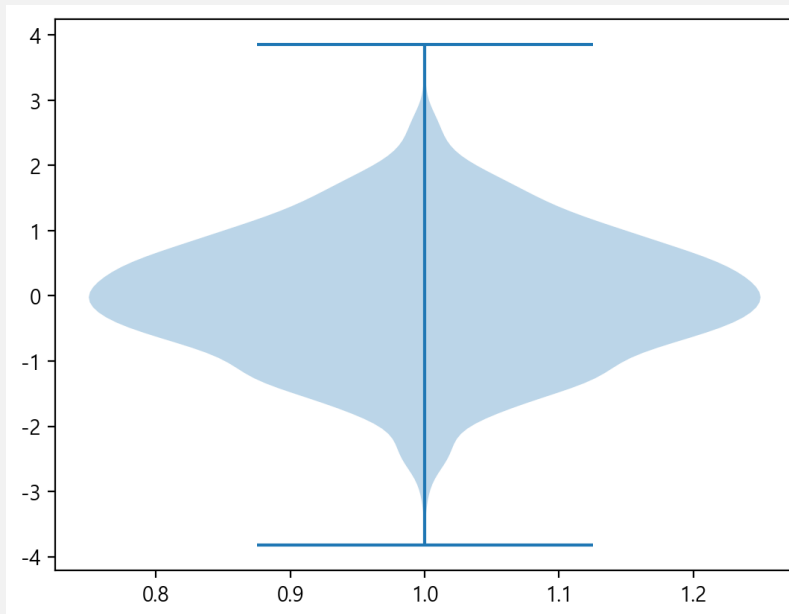
- 상자그림(boxplot)에 데이터 분포의 밀도를 보다 구체적으로 표현하는 방법
- 분포 밀도(kernel density)를 좌우 대칭의 항아리 모양으로 그리는 방식

- 함수 violinplot()

- 정규분포 난수 2000개로 바이올린을 그린 결과
- 인자 vert = False
 - 바이올린을 가로로
- 인자 quantiles=[.25, .5, .75]
 - 사분위수에 선을 그림

```
data = np.random.randn(2000)
plt.violinplot(data);
```

```
plt.violinplot(data, quantiles=[0, .25, .5, .75, 1], vert=False);
```

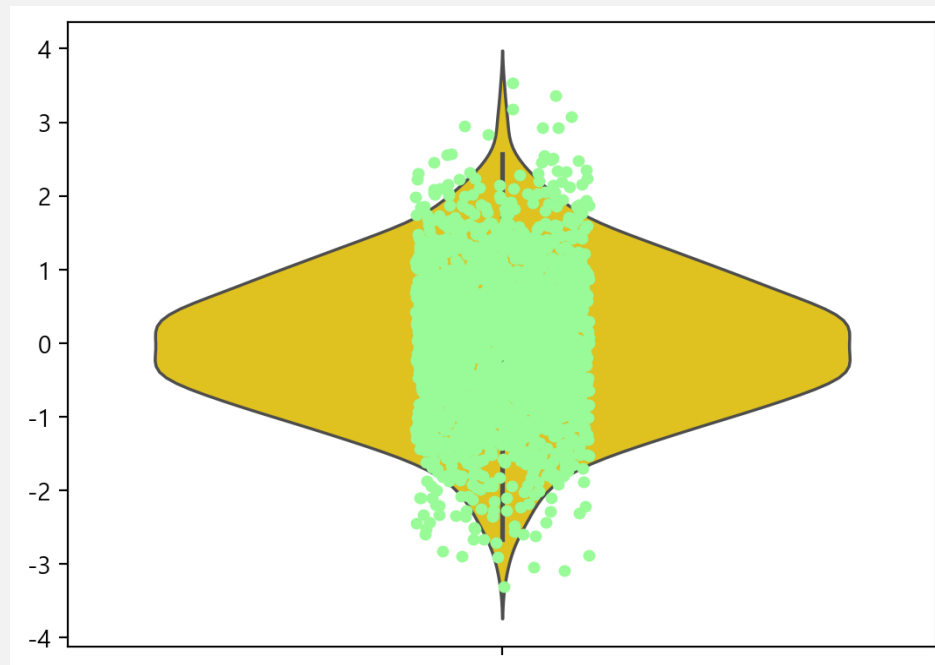


패키지 seaborn의 함수 sns.violinplot()

- 바이올린을 위해 패키지 seaborn의 violinplot()을 사용

- sns.stripplot(data, color='palegreen')를 사용하면 이미 그린 바이올린 플롯에 데이터 분포를 추가

```
import seaborn as sns  
sns.violinplot(data, color='gold');  
sns.stripplot(data, color='palegreen');
```



붓꽃 iris에서 품종 Species에 따라 꽃받침 길이 Sepal.Length의 바이올린 플롯

- 함수 `sns.violinplot()`

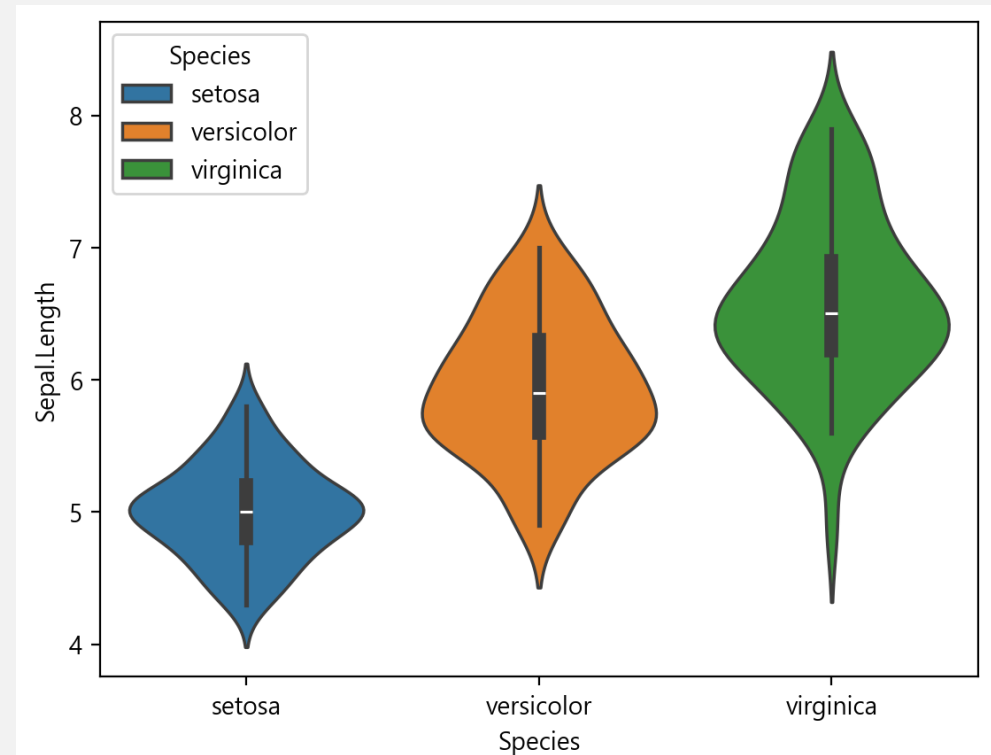
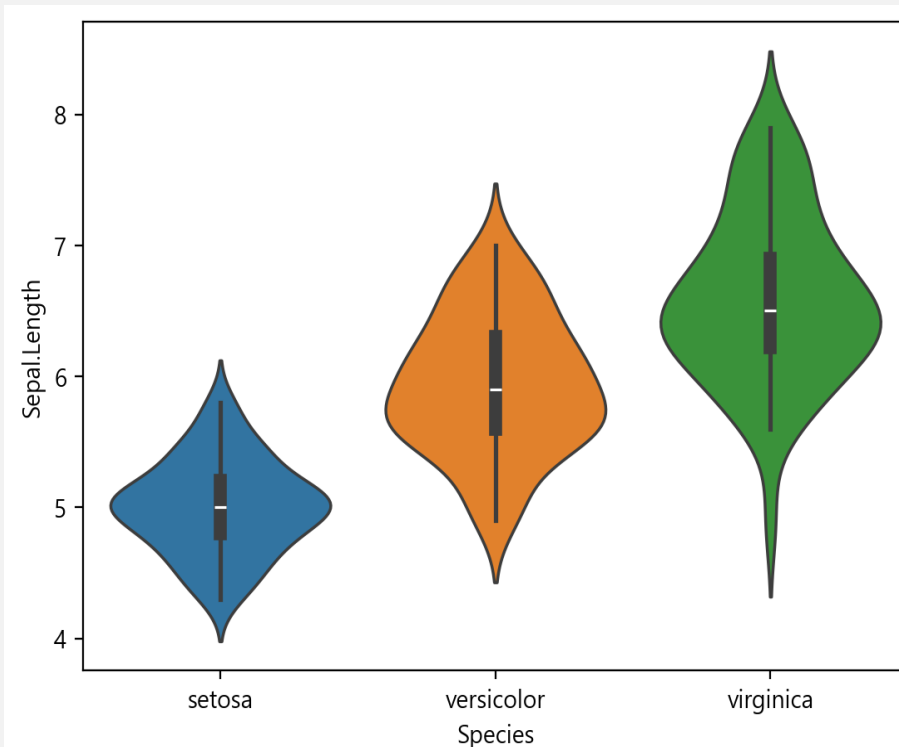
- `x='Species'`
- `y='Sepal.Length'`,
- `hue='Species'`

- 품종에 따라 다른 색상

```
from pydataset import data  
iris = data('iris')
```

```
import seaborn as sns  
sns.violinplot(iris, x='Species', y='Sepal.Length', hue='Species');
```

```
sns.violinplot(iris, x='Species', y='Sepal.Length', hue='Species', legend='full');
```



패키지 seaborn의 함수 sns.pairplot()

- 내장 데이터 mtcars에서 앞 부분 4개의 변수만 추출
 - 4개의 변수 구성 데이터프레임 mincars를 생성
 - 연비(mpg), 실린더 수(cyl), 배기량(displacement) 및 마력(horsepower)

```
print(mtc.columns[:4])
print(mtc.columns[:4].to_list())
```

✓ 0.0s

Index(['mpg', 'cyl', 'displacement', 'horsepower'], dtype='object')
['mpg', 'cyl', 'displacement', 'horsepower']

```
mincars = mtc[mtc.columns[:4].to_list()]
mincars.head(3)
```

✓ 0.0s

	mpg	cyl	displacement	horsepower
Mazda RX4	21.0	6	160.0	110
Mazda RX4 Wag	21.0	6	160.0	110
Datsun 710	22.8	4	108.0	93

```
from pydataset import data
mtc = data('mtcars')
mtc.info()
```

✓ 0.0s

<class 'pandas.core.frame.DataFrame'>
Index: 32 entries, Mazda RX4 to Volvo 142E
Data columns (total 11 columns):
Column Non-Null Count Dtype

0 mpg 32 non-null float64
1 cyl 32 non-null int64
2 disp 32 non-null float64
3 hp 32 non-null int64
4 drat 32 non-null float64
5 wt 32 non-null float64
6 qsec 32 non-null float64
7 vs 32 non-null int64
8 am 32 non-null int64
9 gear 32 non-null int64
10 carb 32 non-null int64
dtypes: float64(5), int64(6)
memory usage: 3.0+ KB

```
from pydataset import data
mtc = data('mtcars')
mincars = mtc.iloc[:, :4]
mincars.head()
```

✓ 0.0s

	mpg	cyl	displacement	horsepower
Mazda RX4	21.0	6	160.0	110
Mazda RX4 Wag	21.0	6	160.0	110
Datsun 710	22.8	4	108.0	93
Hornet 4 Drive	21.4	6	258.0	110
Hornet Sportabout	18.7	8	360.0	175

내장 데이터 mtcars에서 앞 부분 4개의 변수만 추출해 상관관계 행렬 그리기

- 상관관계 시각화 함수 `sns.pairplot()`

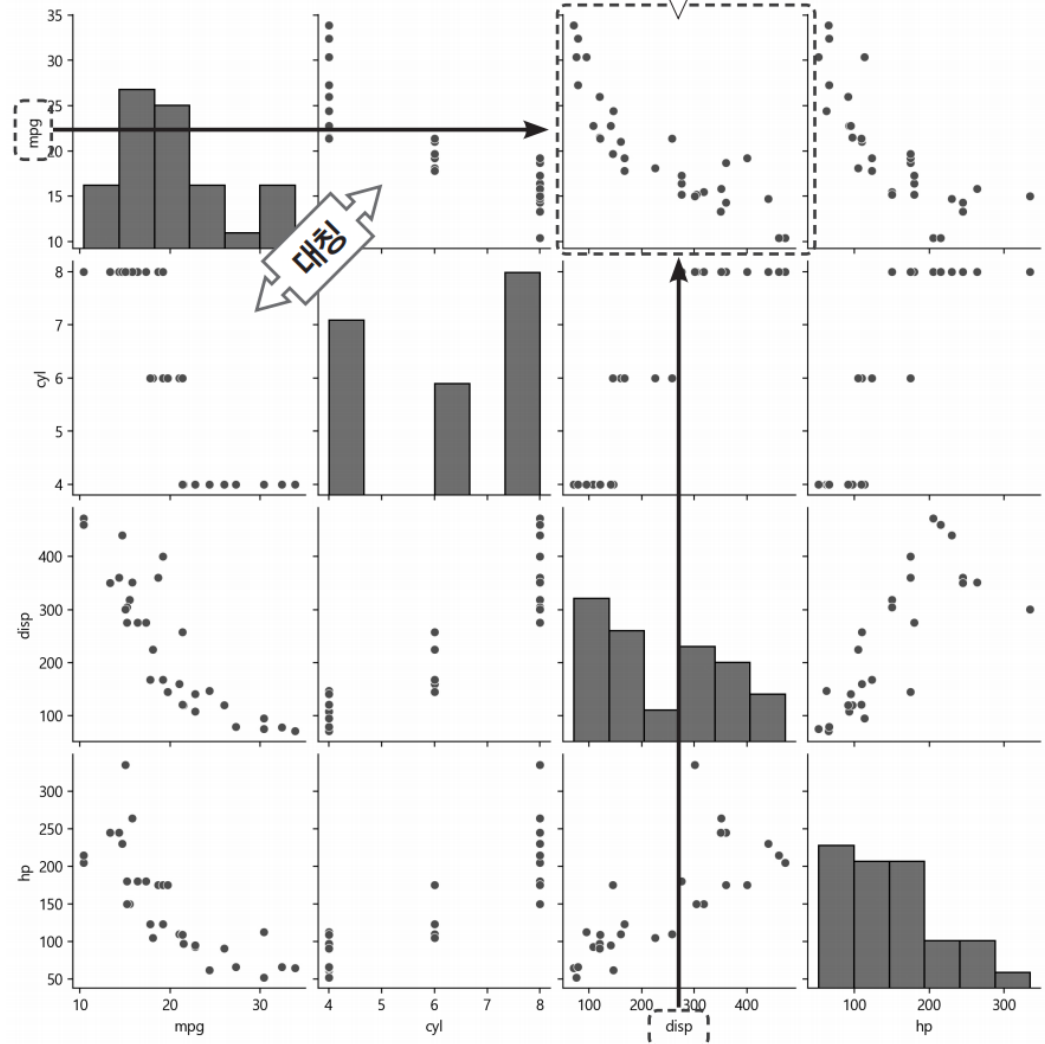
- 두 개 이상의 변수 n개에 대해 모든 가능한 두 변수 간의 관계를 $n \times n$ 테이블 형태로 그리는 상관관계 행렬

- 함수 `pairplot(mtcars)`을 호출

- 간단히 16개의 그림
 - 4×4 행렬
 - 4개의 변수가 x축과 y축의 값
- 행렬의 대각선
 - 4개 변수의 도수분포표가 표시
- 대각선을 중심
 - 우측상단과 좌측하단은 x축과 y축이 바뀐 동일한 산점도가 배치

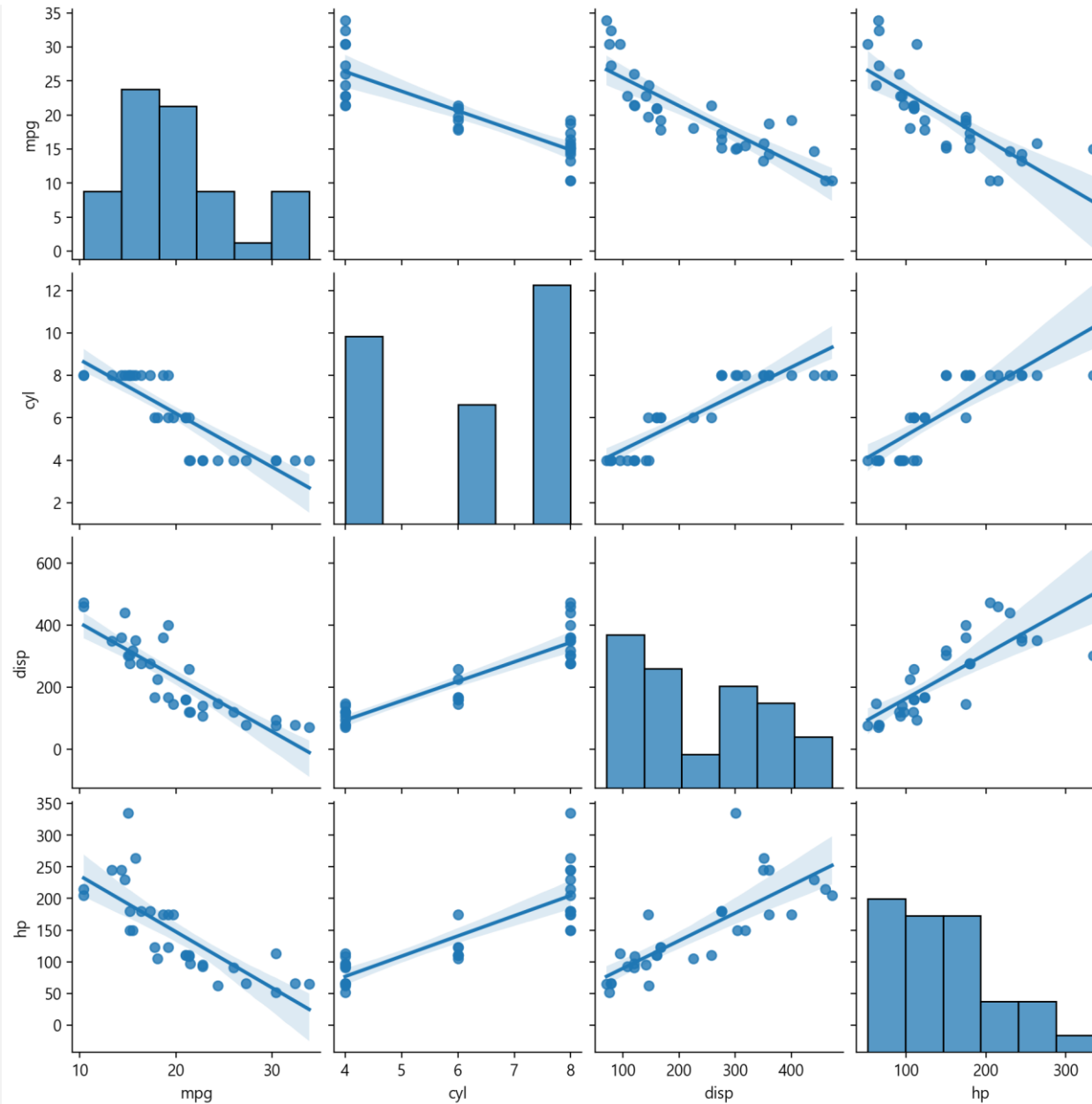
```
import seaborn as sns
sns.pairplot(mtcars);
```

x축은 배기량(dis), y축은 연비(mpg)인 산점도로,
배기량이 크면 연비가 낮아지는 경향을 파악



인자 kind='reg'

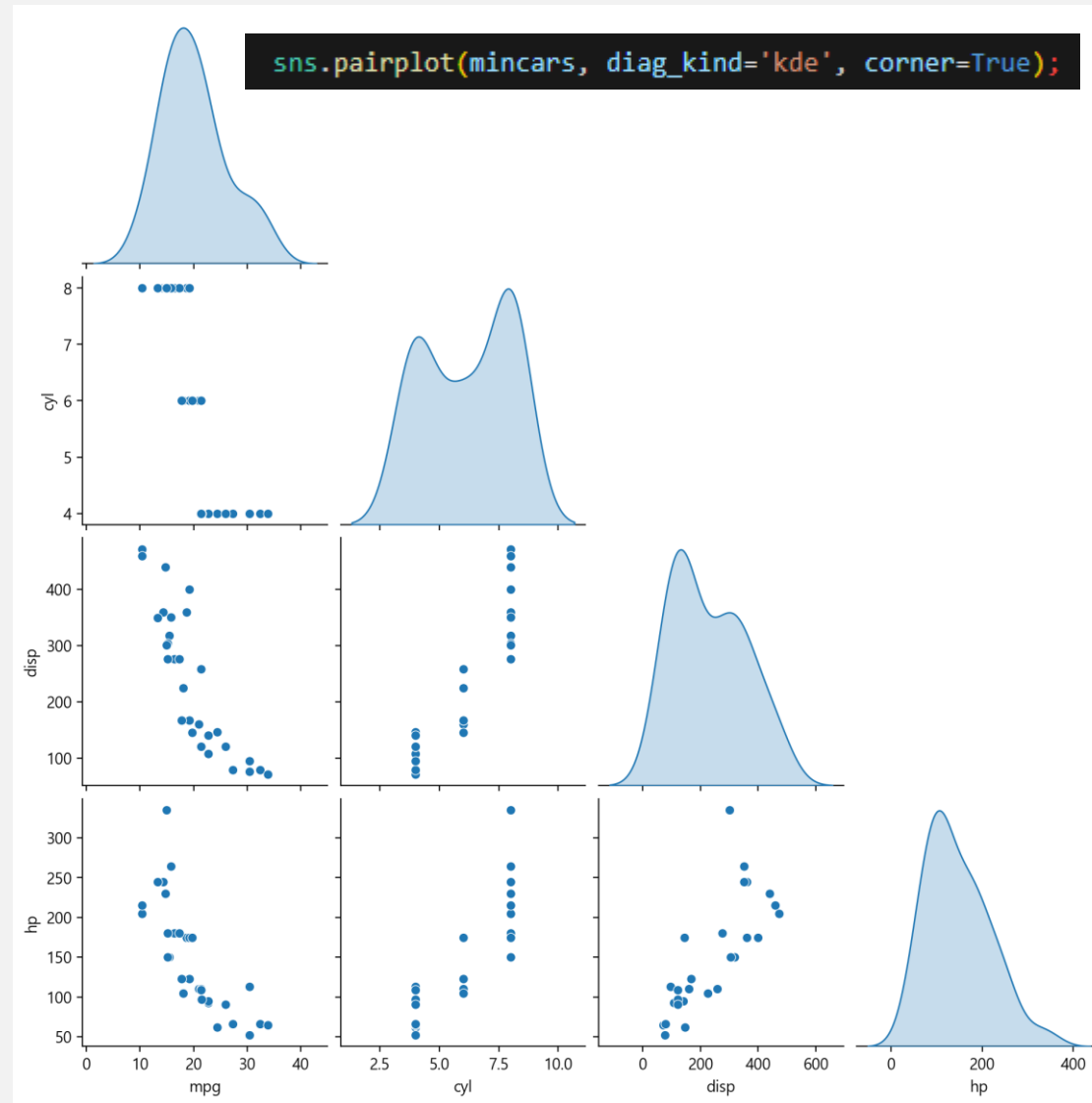
두 변수의 관계를 가장 잘 표현하는 직선(추세선 또는 회귀선)이 포함된 산점도



인자 `diag_kind='kde'`

대각선에 그려지는 도수분포표(hist)를 밀도 그래프인 밀도 그림(kde)

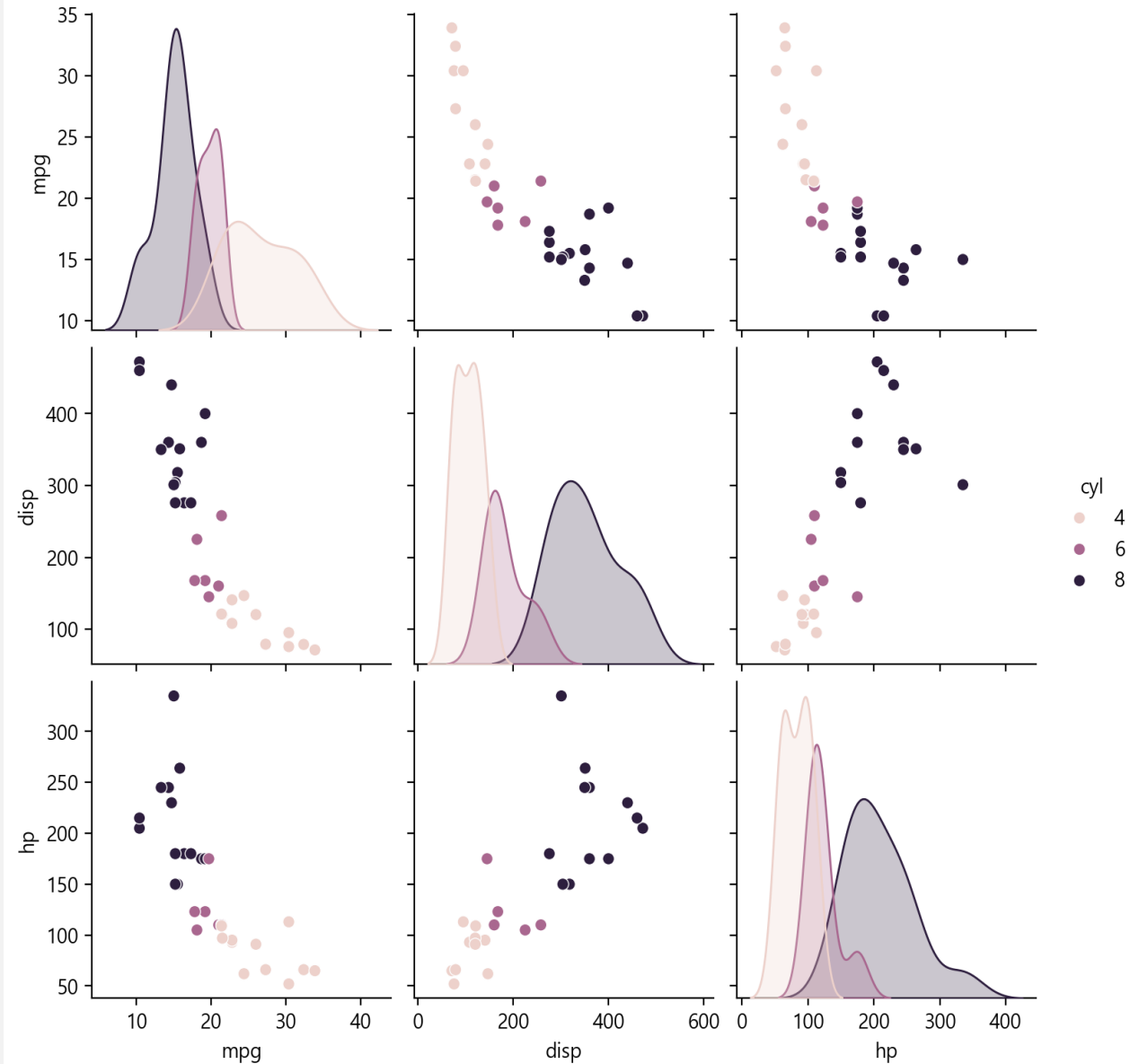
- 인자 `corner=True`를 사용하면 대칭인 부분에서 왼쪽 하단 부분만 그려짐



인자 hue='cyl'

```
sns.pairplot(mincars, hue='cyl');
```

- 지정된 열인 실린더 수 cyl
 - 행과 열에서 제외
 - 각각의 그림에서 색상으로 실린더 수가 구분되어 그려짐
- 대각선 그림이 밀도 그림
 - 실린더 수(cyl)가 많아지면
 - 연비(mpg)가 낮아짐
 - 배기량(dis)과 마력(hp)을 커짐

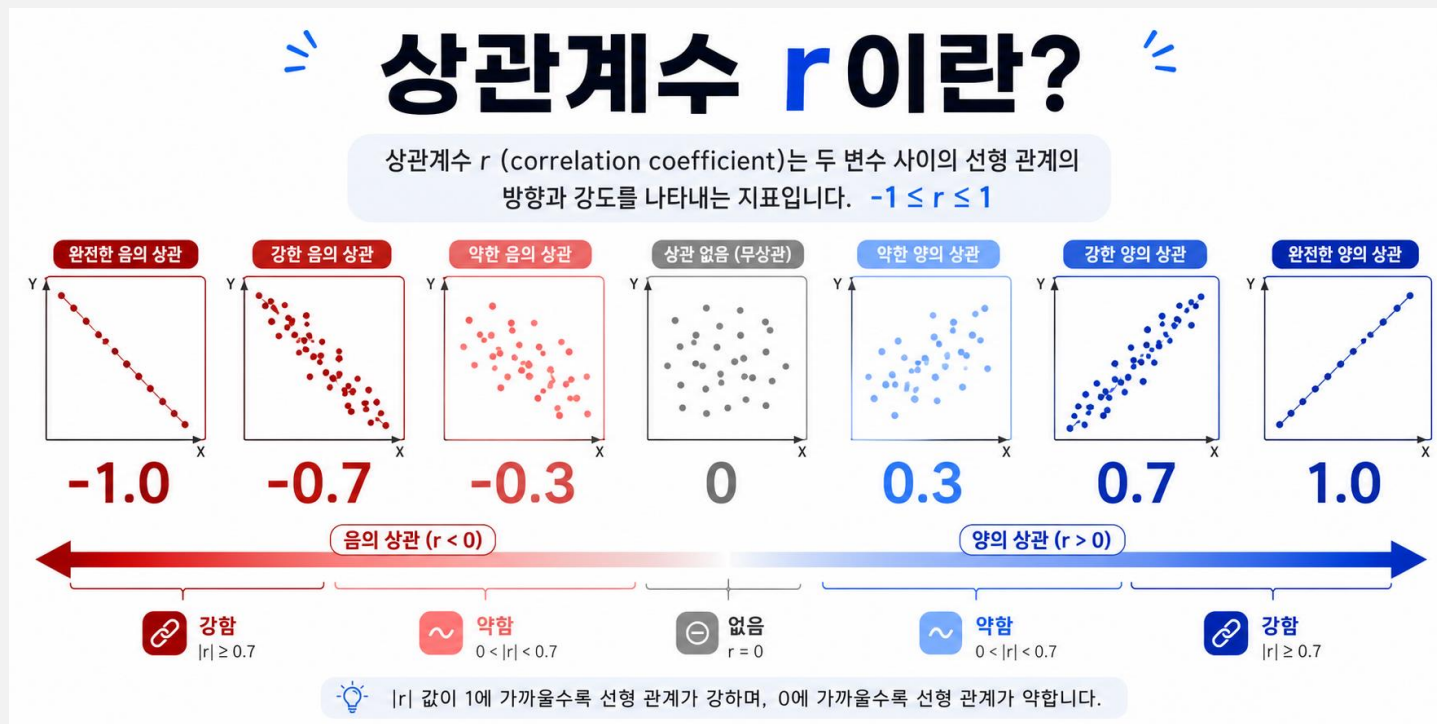


상관계수(correlation coefficient)

상관관계 분석에서 두 변수 간에 선형 관계의 정도를 수량화한 값

• 상관계수 r

- $[-1, 1]$ 실수 값, 다음의 선형관계를 의미
 - r 이 0에 근접할수록 선형 관계가 약해짐
 - 양수 r 값은 양의 상관관계
 - 두 변수 값은 함께 증가하는 경향
 - 음수 r 값은 음의 상관관계
 - 다른 변수 값이 감소할 때 한 변수 값은 증가하는 경향



유명한 붓꽃 데이터 iris를 seaborn에서 가져오기

```
# iris 데이터셋 로드
import seaborn as sns
iris = sns.load_dataset("iris")
iris.info()
```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	sepal_length	150 non-null	float64
1	sepal_width	150 non-null	float64
2	petal_length	150 non-null	float64
3	petal_width	150 non-null	float64
4	species	150 non-null	object

dtypes: float64(4), object(1)
memory usage: 6.0+ KB

df의 메소드 df.corr()

열들 간의 상관관계의 데이터프레임 반환

- 대각선은 1이며 대각선을 기준으로 대칭
 - 품종은 빼고 상관관계수 구하기
 - 꽃잎 너비인 petal_width와 꽃잎 길이인 petal_length의 상관관계는 0.963
 - 매우 관계가 높음

```
# 상관관계 계수 계산
import pandas as pd
pd.set_option('display.precision', 3)
corr_iris = iris[iris.columns[:-1].to_list()].corr()
corr_iris
```

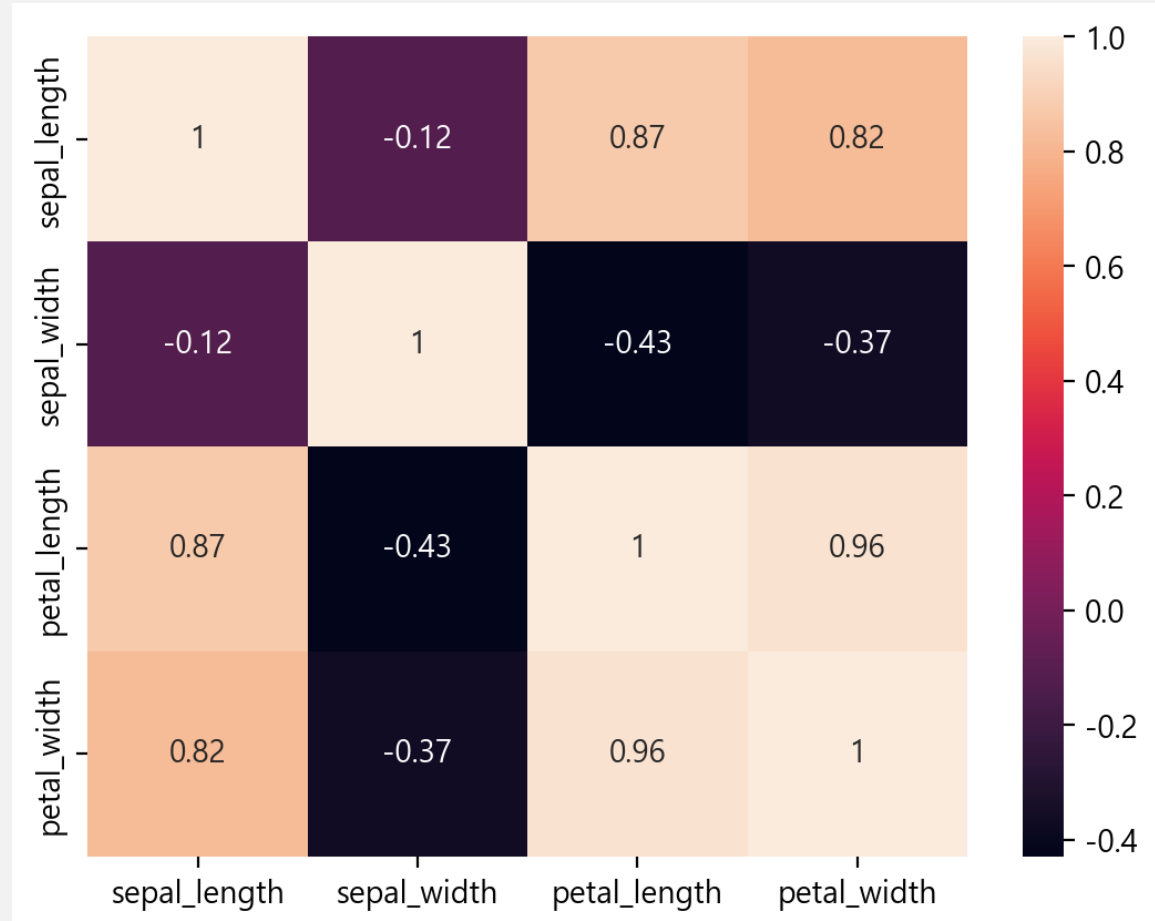
	sepal_length	sepal_width	petal_length	petal_width
sepal_length	1.000	-0.118	0.872	0.818
sepal_width	-0.118	1.000	-0.428	-0.366
petal_length	0.872	-0.428	1.000	0.963
petal_width	0.818	-0.366	0.963	1.000

상관관계 계수를 색상으로 표현

변수들의 관계를 쉽게 파악

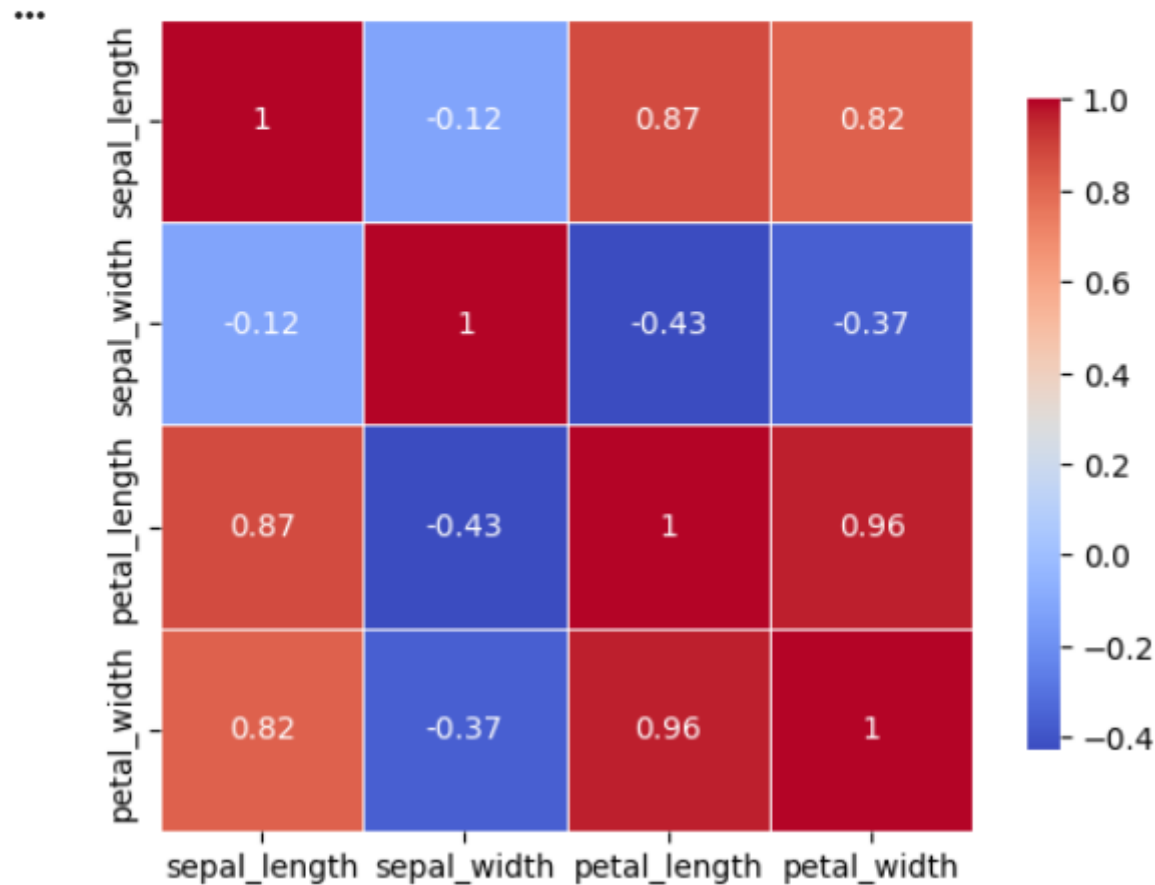
- 값에 따라 색깔이 변하는 행렬 그림이 히트맵(heatmap)
 - 오른쪽 색상 막대(color bar)
 - 상관계수가 1에 가까우면 밝아지고 -1에 가까우면 어두워짐
 - 그러므로 상관계수가 1인 대각선이 가장 밝음

```
sns.heatmap(corr_iris, annot=True);
```



인자 cmap='coolwarm'

```
sns.heatmap(corr_iris,
            annot=True,          # 숫자 표시
            cmap='coolwarm',    # 색상 팔레트
            square=True,        # 정사각형 셀
            linewidths=0.5,     # 셀 테두리
            cbar_kws={'shrink': 0.8}); # 컬러바 크기 조정
```



히트맵을 보기 좋게 좌측 하단의 삼각 행렬만 남도록 수정하기 위한 준비

- 마스크(mask)

- corr_iris와 같은 모양인 4×4 가 모두 0으로 채워진 2차원 행렬 mask를 생성

- np.triu_indices_from(mask)

- mask의 상삼각형에 대한 인덱스를 반환
- 행 인덱스와 열 인덱스

- mask[np.triu_indices_from(mask)] = 1

- mask의 오른쪽 위 대각 행렬을 모두 1로 수정한 mask를 생성

- mask[1:, :-1]

- mask와 상관행렬의 첫 번째 행과 마지막 열을 제거

```
# mask 만들기
import numpy as np
mask = np.zeros_like(corr_iris)
mask
```

✓ 0.0s

Python

```
array([[0., 0., 0., 0.],
       [0., 0., 0., 0.],
       [0., 0., 0., 0.],
       [0., 0., 0., 0.]])
```

```
np.triu_indices_from(mask)
```

✓ 0.0s

Python

```
(array([0, 0, 0, 0, 1, 1, 1, 2, 2, 3]), array([0, 1, 2, 3, 1, 2, 3, 2, 3, 3]))
```

```
# 오른쪽 위 대각 행렬을 1로 바꾸기
mask[np.triu_indices_from(mask)] = 1
mask
```

✓ 0.0s

Python

```
array([[1., 1., 1., 1.],
       [0., 1., 1., 1.],
       [0., 0., 1., 1.],
       [0., 0., 0., 1.]])
```

```
mask_new = mask[1:, :-1] # mask 첫 번째 행, 마지막 열 제거
mask_new
```

✓ 0.0s

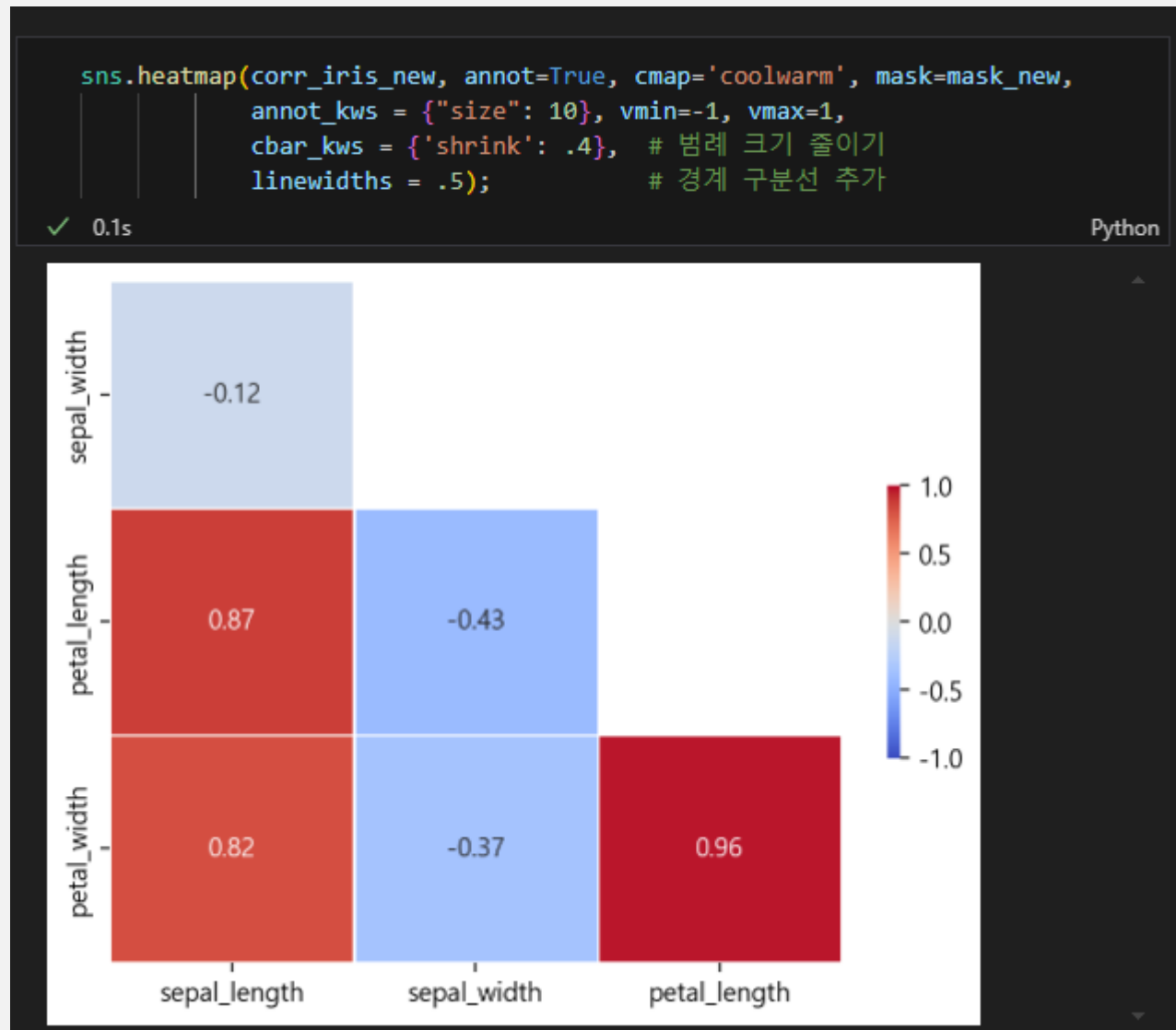
Python

```
array([[0., 1., 1.],
       [0., 0., 1.],
       [0., 0., 0.]])
```

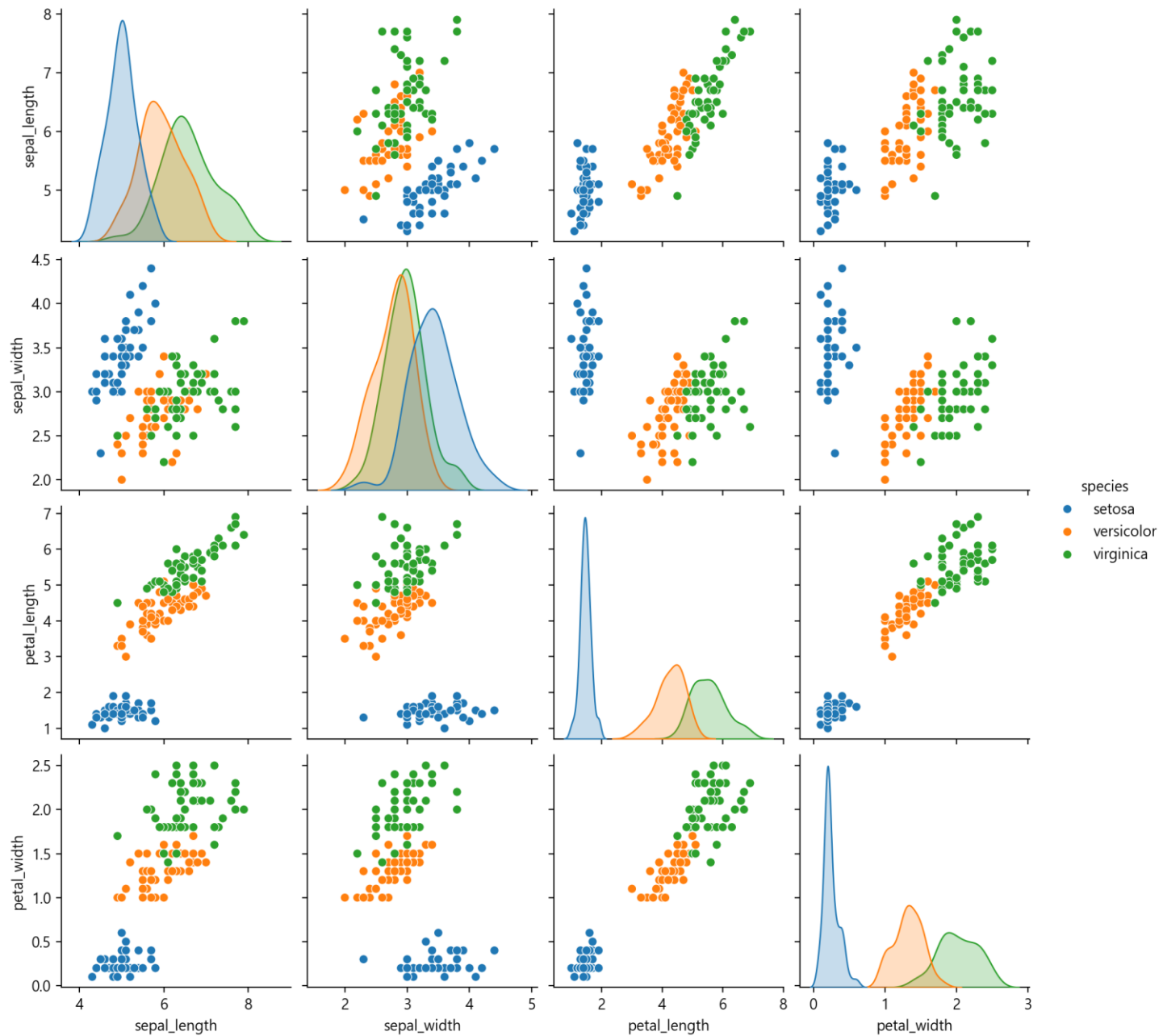
히트맵의 좌측 하단의 삼각 행렬 그리기

3개의 서로 다른 변수 간의 상관관계 계수를 쉽게 파악

- 상관관계 계수 데이터프레임의 첫 번째 행과 마지막 열을 제거한 새로운 데이터프레임 `corr_iris_new` 생성
- 인자 사용
 - `mask=mask_new` 를 사용
 - `mask`가 `True`인 셀을 그리지 않음
 - `cmap='coolwarm'`
 - 양의 상관관계는 붉은 색으로
 - 음의 상관관계는 파란 색으로 표시
 - `cbar_kws = {'shrink': .4}`
 - 칼라바 줄이기



```
sns.pairplot(iris, hue='species')
```

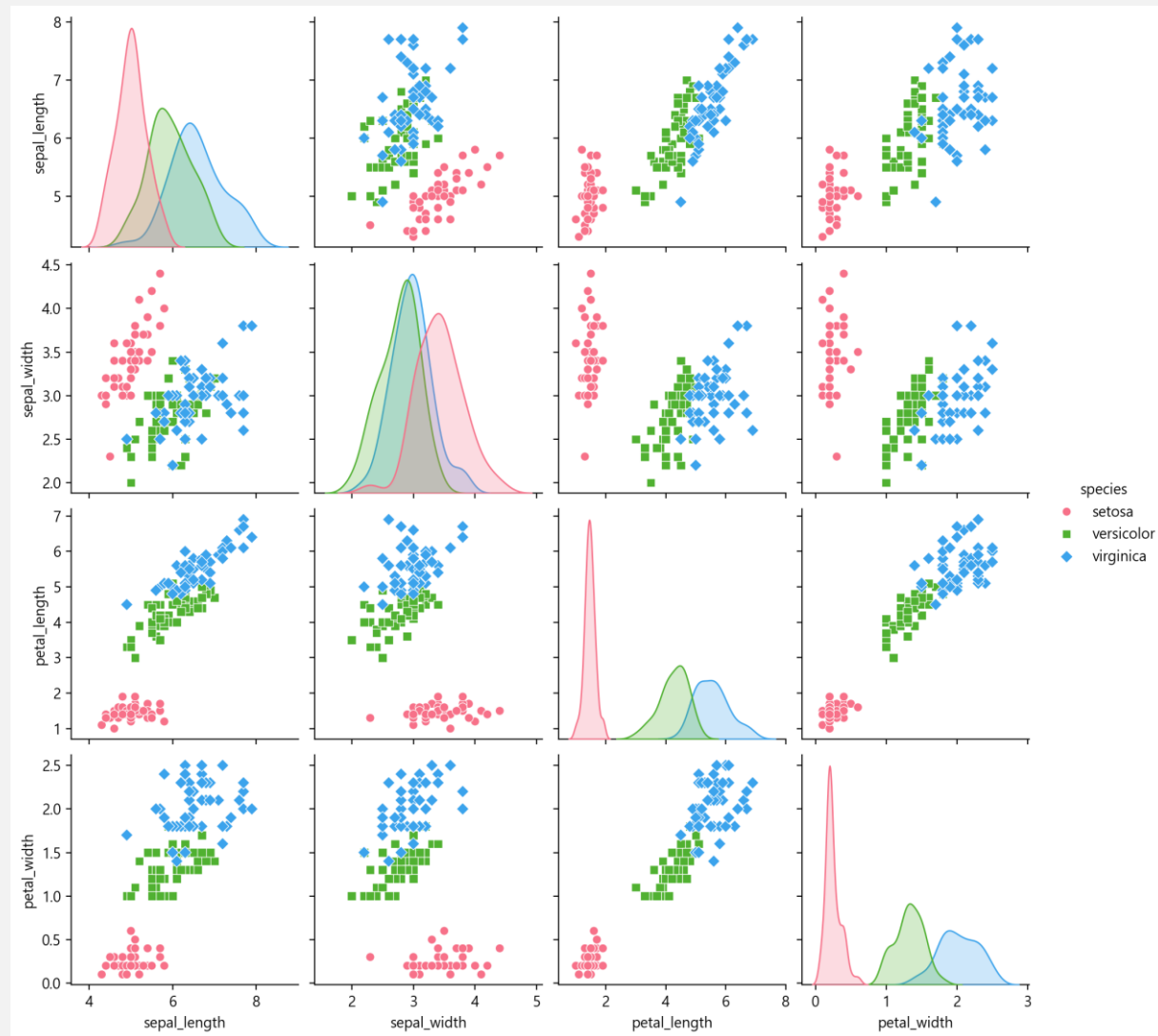


인자 `palette="husl"`, `markers=["o", "s", "D"]`를 사용

품종에 따른 색상과 마커를 수정

```
sns.pairplot(iris, hue="species", palette="husl", markers=["o", "s", "D"]);
```

- **"deep"**
 - 기본값, 무난한 대비
- **"muted"**
 - 채도를 낮춘 색상
- **"bright"**
 - 강한 색 대비
- **"dark"**
 - 어두운 톤
- **"colorblind"**
 - 색각 이상자 고려



좌측 하단을 산점도에 밀도 추정 그림도 추가

함수 `sns.pairplot()`의 반환 객체인 `PairGrid`

- 반환 객체인 `PairGrid` 객체 `g`

- `g.map_lower(sns.kdeplot, ...)` 호출

```
g = sns.pairplot(iris, hue="species",  
                 palette="husl", height=2.0,  
                 markers=["o", "s", "D"]);  
  
g.map_lower(sns.kdeplot, levels=4,  
            color=".2");
```

- `sns.kdeplot`의 관련 인자

- `levels=4`

- 밀도 등고선을 최대 4개 수준으로 나누어 표시
 - 숫자가 클수록 등고선이 더 많아져 세밀한 분포

- `color=".2"`

- 등고선 색상을 지정
 - ".2"는 회색조 표현
 - 값은 문자열 형태의 숫자이며 0에 가까울수록 검정색 1에 가까울수록 흰색
 - `hue` 색상으로 구분 어려움

